



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Laurea Triennale in Informatica

**Simulatore Multi-Core RISC-V con
Estensioni ISA per la Comunicazione tra
Core**

Relatore:

Marco Danelutto

Candidato:

Stella Andolfo

ANNO ACCADEMICO 2025/2026

Indice

1	Introduzione	7
1.1	Obiettivi	7
1.2	Scenario di riferimento	8
1.3	Organizzazione del lavoro	8
2	Tool Background	9
2.1	L'architettura RISC-V:	9
2.1.1	Filosofia e vantaggi del set di istruzioni aperto	9
2.1.2	Il subset RV32I: Registri, PC e gestione della memoria	9
2.1.3	Formati delle istruzioni e modularità (Estensione M)	11
2.2	Il Linguaggio C come strumento di modellazione	13
2.2.1	Perché il linguaggio C: prestazioni e controllo del basso livello	13
2.2.2	Rappresentazione dell'hardware: Array, strutture e puntatori	13
2.2.3	Operazioni bit-a-bit (bitwise) per la decodifica	14
3	Progetto Logico	15
3.1	Architettura Multi-Core: Organizzazione a N core indipendenti	15
3.2	Topologia di Comunicazione	16
3.3	Registri Direzionali	17
3.4	Protocollo di Scambio	17
4	Implementazione	19
4.1	Organizzazione dei Moduli e Gestione della Memoria host	19
4.1.1	Organizzazione dei moduli software	19
4.1.2	Modellazione della memoria RAM simulata	20
4.1.3	Parsing e caricamento dei binari ELF	21
4.2	Rappresentazione dello Stato del core	22
4.2.1	Struttura dati centrale del core	22
4.2.2	Simulazione dei registri architetturali	23
4.2.3	Canali cardinali e comunicazione tra core	23
4.2.4	Inizializzazione dello stato del core	24
4.3	Ciclo di Esecuzione del Singolo Core	24
4.3.1	Fase di fetch	25
4.3.2	Fase di decode	26

4.3.3	Fase di execute	27
4.4	Modello Multi-Core Sincrono e Propagazione Sistolica	28
4.4.1	Problema della concorrenza in simulazione sequenziale	29
4.4.2	Snapshot degli ingressi	29
4.4.3	Esecuzione isolata dei core	30
4.4.4	Propagazione tra core e condizioni al contorno	30
4.5	Estensione dell'ISA: Istruzione Custom 0x0B	31
4.5.1	Integrazione dell'opcode personalizzato	31
4.5.2	Istruzione IN	31
4.5.3	Istruzione OUT	32
4.6	Strumenti di Debug e Osservazione	32
4.6.1	Ispezione dello stato del cluster	32
4.6.2	Monitoraggio dei registri di esecuzione	33
5	Esperimenti e Validazioni	35
5.1	Test funzionali su singola istruzione	35
5.2	Caso di studio: algoritmo di Parallel Sort su rete sistolica	36
5.2.1	Configurazione dell'ambiente sperimentale e codice Assembly	37
5.2.2	Analisi dei log di esecuzione	38
5.2.3	Analisi empirica dei tempi di convergenza	39
6	Conclusioni e Sviluppi Futuri	43
6.1	Sintesi del lavoro svolto	43
6.2	Considerazioni sull'architettura implementata	43
6.3	Sviluppi futuri	44
6.3.1	Estensione della topologia verso reti bidimensionali (Mesh 2D)	44
6.3.2	Integrazione con toolchain di compilazione standard	45
6.3.3	Pipeline multi-stadio e modellazione degli hazard	45
6.3.4	Gestione di valori sentinella e tipi di dato estesi	45
6.3.5	Strumenti di profilazione e visualizzazione	45
A	Codice in C del simulatore RISC-V	47
A.1	Main	47
A.2	Core	50
A.3	Elf	57

Elenco delle figure

2.1	Registri RISC-V	10
2.2	Formato istruzioni RISC-V	12
3.1	Organizzazione logica del cluster multi-core	15
3.2	Astrazione logica delle interfacce di comunicazione del core	17
5.1	Stato finale del core	36
5.2	Ciclo di Clock 0	38
5.3	Ciclo di Clock 11	39
5.4	Ciclo di Clock 40	40

Capitolo 1

Introduzione

1.1 Obiettivi

L'obiettivo principale di questa tesi è la progettazione e l'implementazione di un simulatore multi-core basato sull'architettura RISC-V, realizzato in linguaggio C. Il sistema non si limita all'esecuzione di istruzioni isolate, ma definisce un ambiente di simulazione completo per lo studio del comportamento cooperativo di un cluster di processori interconnessi.

Nel dettaglio, il lavoro si articola nei seguenti obiettivi funzionali.

- Implementazione di un interprete completo per l'ISA RV32I, con gestione del register file a 32 registri, del Program Counter e delle principali operazioni aritmetico-logiche, incluse le estensioni opzionali del set M.
- Estensione dell'ISA mediante introduzione di istruzioni custom. Sfruttando la struttura modulare di RISC-V, sono state definite le primitive `IN` e `OUT`, finalizzate alla comunicazione diretta tra core tramite registri di frontiera associati alle direzioni cardinali (Nord, Sud, Est, Ovest), eliminando la dipendenza dalla memoria condivisa.
- Modellazione del parallelismo mediante esecuzione deterministica su host sequenziale. La sincronizzazione del sistema è ottenuta tramite un ciclo di clock software e un meccanismo di snapshot, che separa in modo esplicito la fase di lettura dallo stato dalla fase di aggiornamento, garantendo coerenza temporale tra i core.
- Validazione del modello attraverso un caso d'uso concreto basato su un algoritmo di ordinamento parallelo (Parallel Sort), utilizzato per analizzare la propagazione dei dati nella rete sistolica e verificare la convergenza del sistema.

Nel complesso, il lavoro dimostra la possibilità di costruire un simulatore compatto ma estensibile, in grado di supportare la sperimentazione di algoritmi paralleli su architetture multi-core a comunicazione diretta.

1.2 Scenario di riferimento

Il contesto attuale dell'architettura dei sistemi di calcolo è caratterizzato da una progressiva adozione di soluzioni hardware specializzate e riconfigurabili. In questo scenario, RISC-V rappresenta una piattaforma di riferimento, in quanto architettura aperta e modulare che consente l'estensione del set di istruzioni senza vincoli proprietari [4]. Tale caratteristica la rende particolarmente adatta alla sperimentazione di nuove soluzioni architetturali e modelli di esecuzione.

Parallelamente, l'evoluzione delle prestazioni computazionali ha evidenziato i limiti del paradigma single-core. L'aumento della frequenza operativa non costituisce più una strategia sostenibile a causa dei vincoli energetici e termici. Per questo motivo, l'attenzione si è spostata verso architetture multi-core e sistemi paralleli, nei quali il carico computazionale viene distribuito su più unità di esecuzione [1].

In questo contesto, la comunicazione tra core rappresenta uno degli elementi critici. L'adozione di memoria condivisa introduce infatti fenomeni di contesa e non determinismo nei tempi di accesso. Ne consegue l'interesse verso modelli alternativi basati su comunicazione diretta tra unità di calcolo, che riducono la dipendenza dalla gerarchia di memoria e rendono più prevedibile il comportamento del sistema.

1.3 Organizzazione del lavoro

La struttura della tesi segue un percorso progressivo che parte dalla definizione dei concetti teorici fino alla descrizione dell'implementazione e alla validazione del sistema.

Il Capitolo 2 analizza il background tecnico di riferimento, introducendo l'architettura RISC-V (RV32I) e le caratteristiche del linguaggio C utilizzato per la modellazione a basso livello del simulatore.

Il Capitolo 3 descrive il progetto logico del sistema, con particolare attenzione alla topologia del cluster multi-core, al modello di comunicazione tra nodi e alla definizione dei registri di interfaccia.

Il Capitolo 4 presenta l'implementazione del simulatore, includendo la gestione del ciclo di esecuzione, la modellazione della memoria e l'introduzione delle istruzioni personalizzate IN e OUT.

Il Capitolo 5 riporta la fase sperimentale, con l'analisi dei test eseguiti e l'applicazione dell'algoritmo di Parallel Sort per la valutazione del comportamento del sistema.

Infine, il Capitolo 6 sintetizza i risultati ottenuti e discute possibili estensioni future del simulatore, sia in termini architetturali che funzionali.

Capitolo 2

Tool Background

Questo capitolo definisce i pilastri tecnologici su cui si basa il simulatore. Vengono analizzate le specifiche dell'architettura RISC-V e le caratteristiche del linguaggio C che lo rendono lo strumento adatto per la modellazione hardware a basso livello.

2.1 L'architettura RISC-V:

2.1.1 Filosofia e vantaggi del set di istruzioni aperto

L'architettura RISC-V nasce con l'obiettivo di fornire un'alternativa aperta e standardizzata alle ISA (Instruction Set Architecture) proprietarie e protette da copyright. Essendo un progetto open-source, permette a progettisti e ricercatori di studiare o realizzare microprocessori senza dover pagare costose licenze di utilizzo [4]. I vantaggi principali di questo approccio sono tre. Innanzitutto la stabilità, dato che il set di istruzioni base è fisso e modificabile solo tramite estensioni; questo garantisce che il software scritto oggi continuerà a funzionare senza problemi anche sui processori sviluppati in futuro. In secondo luogo la semplicità: seguendo la filosofia RISC (Reduced Instruction Set Computer), l'architettura riduce al minimo la complessità logica del circuito eliminando le istruzioni ridondanti. Infine, la modularità consente di avere una struttura organizzata in un modulo base a cui si possono aggiungere liberamente estensioni standard, come quelle per la moltiplicazione o la virgola mobile, o estensioni del tutto personalizzate per scopi specifici.

2.1.2 Il subset RV32I: Registri, PC e gestione della memoria

Il simulatore si sviluppa attorno al subset RV32I, che rappresenta l'architettura di base a numeri interi con registri e indirizzamento a 32 bit. Questo specifico set di istruzioni costituisce il pacchetto minimo e indispensabile per poter garantire l'esecuzione di programmi di tipo generico. La struttura interna del core prevede tre elementi fondamentali che coordinano l'intero flusso dei dati: il Register File, il Program Counter e il sistema di gestione della memoria [4].

Il primo elemento è composto da 32 registri generici (da **x0** a **x31**), ciascuno con una dimensione di 32 bit. Anche se a livello hardware l'architettura permette un utilizzo flessibile di quasi tutti i registri, la convenzione standard ABI (Application Binary Interface) assegna a ognuno di essi un nome simbolico e una funzione specifica per garantire la compatibilità del codice compilato. Il registro **x0** (denominato zero) ha una caratteristica hardware particolare: è cablato internamente a zero (hardwired to zero), cioè qualsiasi tentativo di scrittura su **x0** viene ignorato dall'hardware e la sua lettura restituirà sempre il valore 0. Questo vincolo richiede una gestione software esplicita durante la scrittura dell'interprete nel simulatore. Gli altri registri includono puntatori di controllo e registri di supporto, come **x1** (**ra**) per salvare l'indirizzo di ritorno delle funzioni, **x2** (**sp**) per la gestione dello stack, i registri temporanei (**t0-t6**) per le operazioni intermedie e i registri operando (**a0-a7**) per il passaggio dei parametri alle funzioni [2]. La mappatura completa è schematizzata nella [Figura 2.1](#).

Name	Register Number	Usage
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporaries
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments / return values
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporaries

Figura 2.1: Registri RISC-V

Il secondo elemento è rappresentato dal Program Counter (PC), un registro speciale a 32 bit che contiene l'indirizzo di memoria dell'istruzione in corso di esecuzione. Nello standard RV32I, il PC avanza normalmente di 4 byte alla volta, poiché ogni istruzione ha una lunghezza fissa di 32 bit. Il flusso può essere modificato solo da istruzioni di salto condizionato (branch) o incondizionato (jump), le quali calcolano il nuovo indirizzo tramite un meccanismo di indirizzamento relativo rispetto alla posizione corrente del PC. [4]

Infine, la gestione della memoria adotta un'architettura di tipo Load/Store, questo implica che le operazioni logico-aritmetiche possano essere eseguite esclusivamente su dati già caricati all'interno dei registri del core. L'accesso alla memoria RAM esterna non avviene mai direttamente tramite le istruzioni di calcolo, ma è

consentito esclusivamente tramite istruzioni dedicate di lettura (come `lw`, `lh`, `lb`) e scrittura (come `sw`, `sh`, `sb`). L'architettura RISC-V implementa uno spazio di memoria indirizzabile al singolo byte (byte-addressable memory), e poiché una parola dati a 32 bit (word) è composta da 4 byte, l'indirizzamento procede per multipli di 4. Di conseguenza, dato che tutte le istruzioni hanno una lunghezza fissa di 32 bit, anche gli indirizzi in memoria risultano allineati a multipli di 4 byte.[4].

2.1.3 Formati delle istruzioni e modularità (Estensione M)

Formati delle istruzioni

L'architettura RISC-V è progettata per essere il più semplice e regolare possibile, questa semplicità si riscontra anche nei formati delle istruzioni: la struttura dei comandi a 32 bit è studiata in modo che i campi più importanti, come i registri sorgente (`rs`) e destinazione (`rd`), si trovino sempre nelle stesse posizioni. In questo modo, il circuito hardware che deve decodificare le istruzioni diventa molto più semplice e veloce, poiché i segnali possono essere prelevati da posizioni fisse [4]. Lo standard definisce sei formati base di istruzione: R, I, S, B, U e J [4].

- **R (Register-to-register)**: Utilizzato per le operazioni logico-aritmetiche pure tra registri interni (come somme, sottrazioni e l'intera Estensione M). I 32 bit sono divisi in: opcode (7 bit), rd (5 bit per il registro di destinazione), funct3 (3 bit di controllo), rs1 (5 bit per il primo registro sorgente), rs2 (5 bit per il secondo registro) e funct7 (7 bit finali di controllo).
- **I (Immediate)**: Impiegato per le operazioni che usano una costante numerica immediata (come ADDI) e per le istruzioni di lettura dalla memoria (Load). Sostituisce il campo funct7 e rs2 con un valore immediato a 12 bit con segno.
- **S (Store)**: Usato per le istruzioni di scrittura in memoria (come SW). Non avendo un registro di destinazione rd, sfrutta quei bit, insieme ai bit posizionati in cima all'istruzione, per comporre un valore immediato da 12 bit diviso in due parti.
- **B (Branch)**: Dedicato ai salti condizionati (come BEQ o BNE). La struttura è simile al formato S, ma i bit del valore immediato vengono mescolati (rimescolamento dei bit) a livello hardware per facilitare il calcolo immediato dell'indirizzo di salto relativo al Program Counter.
- **U (Upper Immediate)**: Utilizzato per caricare costanti grandi a 20 bit nei bit più significativi di un registro, come le istruzioni LUI e AUIPC. Il formato contiene solo l'opcode, il registro rd e l'immediato da 20 bit. Non prevede registri sorgente, poiché opera esclusivamente su costanti [?].
- **J (Jump)**: Pensato per i salti incondizionati come JAL, contiene un immediato da 20 bit i cui bit sono rimescolati (*bit scrambling*) per ottimizzare i collega-

menti elettrici dei bus hardware e minimizzare il numero di multiplexer nel circuito di decodifica.

I formati delle istruzioni sono riassunti nella [Figura 2.2](#)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Figura 2.2: Formato istruzioni RISC-V

Estensione M

L'Estensione M si integra perfettamente in questa filosofia di modularità sfruttando esclusivamente il formato R-type. Poiché il set base RV32I non include moduli hardware per calcolare prodotti e divisioni, senza questa estensione il core dovrebbe fare i calcoli via software, rallentando drasticamente l'esecuzione dei programmi [2]. Per garantire che il simulatore sia compatibile con i compilatori standard nel codice sono state implementate tutte e otto le istruzioni dell'Estensione M.

Moltiplicazioni (campo funct7 = 0000001):

- **mul**: esegue la moltiplicazione standard a 32 bit (salva la parte bassa del risultato).
- **mulh**, **mulhu**, **mulhsu**: calcolano il prodotto a 64 bit e ne estraggono i 32 bit superiori, gestendo operandi con segno, senza segno o misti

Divisioni e Resti (campo funct7 = 0000001):

- **div** e **divu**: eseguono la divisione intera (con o senza segno), troncando la parte decimale.
- **rem** e **remu**: calcolano il resto della divisione (modulo), fondamentale per la gestione dei vettori e dei flussi nel nostro algoritmo.

2.2 Il Linguaggio C come strumento di modellazione

2.2.1 Perché il linguaggio C: prestazioni e controllo del basso livello

Per sviluppare un simulatore hardware, la scelta del linguaggio di programmazione è fondamentale. Nel progetto è stato scelto il linguaggio C perché si colloca a metà strada tra i linguaggi ad alto livello e il linguaggio macchina. Il C non introduce meccanismi pesanti di gestione automatica, garantendo un'esecuzione estremamente rapida e un consumo di memoria minimo.

Nel caso di un simulatore multi-core, le prestazioni sono un requisito importante: l'intera struttura deve simulare il comportamento di più processori che lavorano in contemporanea, eseguendo migliaia di istruzioni assembly al secondo. La velocità del C permette di ridurre i tempi di calcolo durante i test di algoritmi complessi. Inoltre, questo linguaggio offre un controllo totale sulla memoria e permette di manipolare i singoli bit, un'operazione indispensabile quando si deve riprodurre il comportamento dei circuiti digitali e dei bus di comunicazione

2.2.2 Rappresentazione dell'hardware: Array, strutture e puntatori

All'interno del simulatore, i componenti fisici di un core RISC-V devono essere tradotti in strutture dati astratte, un passaggio dall'hardware al software che avviene in modo lineare attraverso l'uso di costrutti standard del linguaggio C. I registri generici e il Program Counter vengono modellati concettualmente come array e variabili di tipo intero a 32 bit senza segno `uint32_t`, una scelta di tipo dato che garantisce che ogni registro occupi in memoria esattamente i quattro byte previsti dalle specifiche ufficiali dell'architettura reale.

Allo stesso modo, la memoria RAM esterna e condivisa viene simulata allocando un ampio array di byte `uint8_t`, dove l'uso dei puntatori e della relativa aritmetica permette di riprodurre fedelmente la gestione degli indirizzi fisici; in questo modo, ogni volta che l'interprete esegue un'istruzione di salto o di accesso ai dati, lo spostamento all'interno dello spazio di memoria avviene manipolando direttamente gli indici dell'array.

Infine, per gestire il parallelismo a livello di sistema, l'intero stato computazionale di un singolo core, comprendente il banco dei registri, il Program Counter e le variabili di stato, viene incapsulato all'interno di una struttura dati logica (`struct`). Questa organizzazione modulare permette di rappresentare l'intera architettura del cluster multi-core semplicemente come un insieme ordinato di tali strutture, in cui ogni singolo elemento descrive un nodo di calcolo indipendente e isolato all'interno della rete complessa. I dettagli e i costrutti software specifici legati alla dichiarazione

di queste componenti verranno approfonditi nel capitolo dedicato allo sviluppo del codice dell'applicazione.

2.2.3 Operazioni bit-a-bit (bitwise) per la decodifica

La fase più delicata di un simulatore è la decodifica (Decode), ovvero il momento in cui l'interprete legge l'istruzione a 32 bit estratta dalla memoria e ne isola i campi operativi. In un vero core questo compito è svolto da reti logiche combinatorie; nel simulatore il comportamento viene riprodotto tramite gli operatori bitwise del linguaggio C (come l'AND logico `&`, l'OR logico `|` e gli scorrimenti direzionali `»` e `«`).

Quando il simulatore riceve una parola a 32 bit, utilizza delle maschere di bit per isolare i campi definiti dallo standard RISC-V. Ad esempio, per estrarre l'opcode, viene eseguita un'operazione di AND logico con una maschera fissa per azzerare i bit superiori e isolare i 7 bit inferiori. Per identificare i registri sorgente `rs1` o `rs2`, i bit della parola vengono fatti scorrere verso destra per allinearli e poi isolati tramite maschere a 5 bit.

Un altro punto critico in cui gli operatori bitwise si rivelano fondamentali è l'estensione del segno (sign extension) per i valori immediati. Poiché questi valori intermedi sono spesso più piccoli di 32 bit (ad esempio 12 bit nel formato I-type), il simulatore deve copiare il bit di segno più significativo su tutti i restanti bit di sinistra per mantenere corretto il valore matematico all'interno del registro a 32 bit. Questa operazione viene scritta combinando maschere e scorrimenti veloci, imitando esattamente il comportamento dei collegamenti fisici all'interno di un'architettura hardware reale, i cui dettagli implementativi specifici verranno approfonditi nel capitolo dedicato all'implementazione del software.

Capitolo 3

Progetto Logico

3.1 Architettura Multi-Core: Organizzazione a N core indipendenti

La scelta di sviluppare un'architettura multi-core nasce dall'esigenza di velocizzare le operazioni di ordinamento distribuendo il calcolo su una catena di più unità che lavorano in parallelo. Il simulatore è strutturato come un cluster omogeneo composto da un numero N di core computazionali. Ciascun nodo di questa struttura è un core autonomo dotato del proprio Program Counter (PC) privato e di un banco di trentadue registri generici completamente isolato dagli altri, come si può notare nella [Figura 3.1](#).

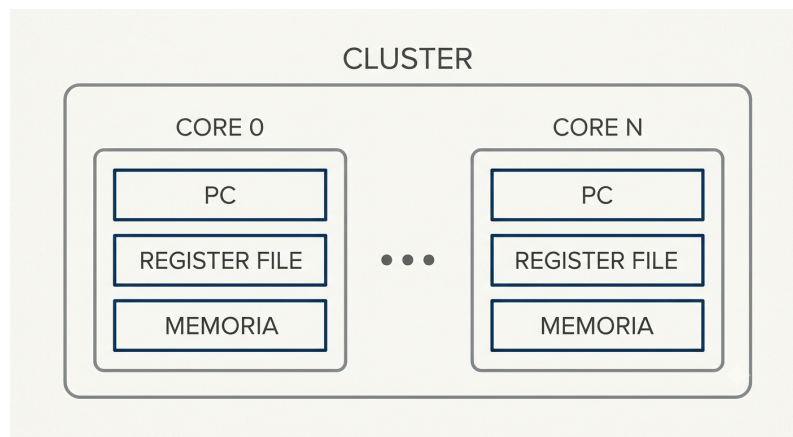


Figura 3.1: Organizzazione logica del cluster multi-core

Dal punto di vista del software, tutti i core del sistema eseguono contemporaneamente lo stesso identico codice di confronto, applicando le istruzioni in base al flusso di dati che ricevono dal nodo precedente. L'indipendenza dei banchi dei registri e dei Program Counter permette a ciascun core di gestire i propri confronti interni senza

interferire con le risorse degli altri nodi. La coordinazione globale non è lasciata a logiche asincrone o a complessi protocolli di stallo della pipeline; l'architettura si configura invece come un sistema parallelo sincrono in cui l'avanzamento dei Program Counter è strettamente guidato da un clock globale gestito dal simulatore. A ogni ciclo, tutti i core attivi compiono un singolo passo di esecuzione in modo simultaneo, garantendo una progressione temporale omogenea lungo l'intera catena di calcolo. La scalabilità dell'intero sistema è legata al parametro configurabile N , che definisce la lunghezza massima della pipeline di ordinamento. Nell'implementazione corrente, il valore massimo di N è fissato a 4 tramite la macro di compilazione `#define N 4` nel file `main.c`. Questa scelta semplifica l'allocazione statica del cluster durante il prototipo; una futura versione potrebbe rendere tale parametro configurabile a runtime tramite allocazione dinamica. Nel software del simulatore, questo valore può essere modificato liberamente prima di avviare l'esecuzione per studiare il comportamento dell'architettura al variare della potenza di calcolo disponibile. Un numero N di core permette di ordinare in modo efficiente un vettore composto da un numero equivalente di elementi. Aumentare la dimensione del cluster consente quindi di gestire sequenze numeriche più lunghe, sfruttando al massimo il parallelismo sincrono dell'hardware e riducendo il tempo complessivo necessario a completare l'intera catena di confronti.

3.2 Topologia di Comunicazione

La scelta della topologia di rete determina l'organizzazione dei canali di comunicazione e condiziona direttamente i percorsi fisici che i dati possono seguire all'interno del sistema. Nel simulatore è stata esclusa l'adozione di un bus condiviso o di una memoria globale centralizzata a cui tutti i core possano accedere liberamente; se tutte le unità tentassero di comunicare sulla stessa linea, il sistema subirebbe immediati colli di bottiglia dovuti alla saturazione del canale. L'architettura sfrutta invece una topologia di interconnessione punto-a-punto basata sugli assi cardinali (Nord, Sud, Est e Ovest). All'interno di questa griglia, i canali fisici collegano ogni singolo core esclusivamente con i suoi quattro vicini immediati, eliminando la necessità di logiche complesse di routing e garantendo uno scambio dati locale e privo di collisioni.

Questa specifica disposizione hardware è stata progettata per supportare in modo efficiente i flussi di dati sequenziali tipici del calcolo a pipeline. La presenza di collegamenti diretti e dedicati lungo gli assi cardinali permette di configurare i core come una catena di calcolo lineare o geometrica, in cui il Core 0 funge da unico punto di ingresso per le informazioni provenienti dall'esterno. La rete permette così a ciascun nodo di ricevere dati dalla propria porta di ingresso e di inoltrare istantaneamente i valori in eccedenza verso il core adiacente tramite la porta d'uscita opposta. Questa struttura direzionale permette al cluster di comportarsi come un array sistolico, dove l'infrastruttura di rete non funge da semplice mezzo di trasporto passivo, ma mappa fisicamente lo spostamento degli elementi un passo alla volta,

coordinando perfettamente la distribution spaziale dei dati con l'avanzamento dei cicli di clock globali. Nella configurazione lineare attualmente implementata, i canali attivi sono esclusivamente quelli orientati lungo l'asse Est-Ovest. I canali Nord e Sud sono definiti nell'interfaccia logica del core (si veda la [Figura 3.2](#)) per consentire future estensioni verso topologie bidimensionali (Mesh 2D), ma non partecipano alla propagazione dei dati nel prototipo corrente.

3.3 Registri Direzionali

Per permettere al software in esecuzione sui singoli core di interagire con la rete fisica descritta in precedenza, l'architettura mappa le connessioni esterne direttamente all'interno dello spazio dei registri del processore. Ogni core non ha bisogno di conoscere la struttura dell'intero cluster, ma vede il mondo circostante attraverso quattro interfacce logiche dedicate, denominate in base agli assi cardinali: Nord, Sud, Est e Ovest. Ciascuna di queste direzioni è composta internamente da una coppia di registri specifici: un registro di ingresso (Input) e un registro di uscita (Output), come illustrato nella [Figura 3.2](#)

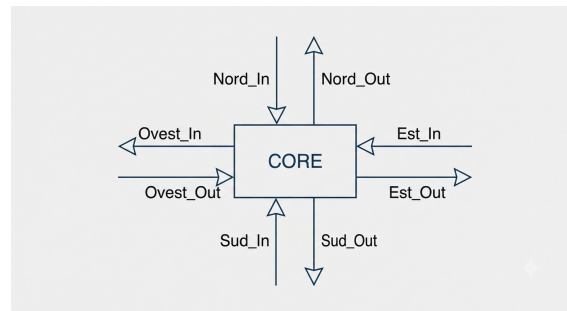


Figura 3.2: Astrazione logica delle interfacce di comunicazione del core

Questi registri speciali fungono da veri e propri buffer di frontiera tra la pipeline interna del core e i canali della griglia. Quando un core deve inviare un dato verso il nodo adiacente, scrive il valore nel registro di uscita associato a quella specifica direzione. Specularmente, il core vicino vedrà comparire quel bit pattern nel proprio registro di ingresso orientato verso la direzione di provenienza. Questa astrazione logica semplifica drasticamente la gestione dell'hardware, poiché trasforma lo scambio di dati tra processori distinti in una semplice operazione di lettura o scrittura su un registro locale, eliminando la necessità di accedere a memorie RAM condivise o di gestire complessi protocolli di rete software.

3.4 Protocollo di Scambio

Il corretto funzionamento di un'architettura in cui i dati scorrono sequenzialmente lungo i nodi richiede un controllo rigoroso sulla temporizzazione delle comunicazioni.

La semplice presenza dei registri direzionali di ingresso e uscita non è infatti sufficiente a garantire l'integrità del calcolo: se un core modificasse il proprio registro di uscita mentre il core adiacente sta leggendo lo stesso canale, si verificherebbero fenomeni di instabilità o corruzione del dato. Per risolvere questa problematica, il simulatore esclude l'adozione di protocolli hardware di handshake o cicli di stall software, implementando un modello di trasferimento dati strettamente sistolico basato sul campionamento a ogni ciclo di clock. Come implementato nella logica di controllo del cluster, ogni ciclo di clock globale viene suddiviso in tre fasi sequenziali e atomiche:

1. Fase di Snapshot: All'inizio del ciclo di clock, lo stato di tutti i registri di ingresso (*IN*) del cluster viene memorizzato e temporaneamente isolato in una struttura dati di supporto. Questo garantisce che i core leggano valori stabili e non influenzati da variazioni repentine sui canali durante la successiva elaborazione.
2. Fase di Calcolo Parallelo: Ciascun core esegue in parallelo una singola istruzione di calcolo o di confronto della pipeline. Qualsiasi operazione di lettura o scrittura (mediata dall'interfaccia logica dei registri direzionali) attinge dallo snapshot iniziale e aggiorna i registri di uscita (*OUT*) locali, senza interferire immediatamente con lo stato dei nodi vicini.
3. Fase di Propagazione: Al termine del passo di calcolo, il simulatore aggiorna i collegamenti di rete punto-a-punto prima dell'inizio dello step successivo. I valori depositati nei registri di uscita di ciascun core vengono copiati e propagati direttamente nei registri di ingresso del core adiacente lungo l'asse lineare configurato (da Est a Ovest).

Questo meccanismo a tre fasi assicura che il dato trasferito sia valido e stabile per l'intera durata del ciclo successivo. La coordinazione temporale è quindi intrinseca alla struttura stessa dell'infrastruttura: il simulatore non si limita a trasferire passivamente i bit, ma mappa lo spostamento spaziale degli elementi a ogni fronte di clock, realizzando una pipeline sincrona nativa dove l'avanzamento dei dati è perfettamente sincronizzato con l'evoluzione temporale del cluster.

Capitolo 4

Implementazione

Questo capitolo descrive la progettazione software e lo sviluppo del simulatore a livello di ciclo macchina. L'obiettivo centrale del framework è tradurre in un ambiente virtuale eseguibile le specifiche hardware dell'architettura RV32I e la topologia di interconnessione della catena multi-core sistolica. Lo studio analizza la struttura dei moduli software, l'organizzazione dello stato interno della CPU e la scomposizione delle tre fasi del ciclo macchina (fetch, decode ed execute). Viene inoltre esaminato il meccanismo di sincronizzazione temporale per la propagazione dei dati tra core adiacenti, illustrando l'estensione dell'Instruction Set Architecture (ISA) tramite opcode personalizzati e gli strumenti di diagnostica integrati per il monitoraggio del cluster.

4.1 Organizzazione dei Moduli e Gestione della Memoria host

Per realizzare un simulatore efficiente è necessario risolvere tre problemi: suddividere il codice sorgente in componenti logiche indipendenti, mappare lo spazio di indirizzamento della memoria garantendo l'isolamento tra i vari core, e implementare un meccanismo di parsing in grado di decodificare i file binari per inizializzare il sistema. L'architettura del software risponde a queste esigenze separando le funzioni di calcolo della CPU dalle funzioni di interfacciamento con l'host, sotto il controllo di un modulo centrale che sincronizza l'avanzamento temporale dell'intero cluster.

4.1.1 Organizzazione dei moduli software

Il codice sorgente è suddiviso in cinque file principali, organizzati per garantire la modularità del framework e una chiara separazione delle competenze (Separation of Concerns):

- `core.h` e `core.c`: rappresentano il motore computazionale del singolo processore. Definiscono la struttura dati della CPU, il banco dei registri generali, i

registri di comunicazione cardinale e l'intera logica del ciclo macchina (fetch, decode, execute). Questo modulo è indipendente dalla topologia del cluster e si occupa esclusivamente di far avanzare un singolo core alla volta.

- `elf.h` e `elf.c`: costituiscono il sottosistema di caricamento del firmware. Contengono le strutture e le routine necessarie a interpretare i file binari in formato standard ELF32 generati dalla catena di compilazione, infine estraggono le sezioni di codice e dati per scriverle direttamente all'interno della memoria simulata.
- `main.c`: rappresenta il modulo di controllo, inoltre gestisce l'interfaccia a riga di comando per acquisire i parametri di configurazione e implementa la logica del clock globale (`run_multi_core`) che dirige l'esecuzione sincrona dell'intero cluster.

4.1.2 Modellazione della memoria RAM simulata

L'architettura RISC-V standard prevede uno spazio di indirizzamento a 32 bit orientato al singolo byte (byte-addressable). Nel simulatore, per ottimizzare le prestazioni ed evitare costose operazioni di ricomposizione dei byte a runtime, la RAM locale di ciascun core è modellata come un array lineare di parole a 32 bit. La coerenza tra il caricamento dei dati e l'allineamento della memoria è garantita da due macro dislocate nei file di configurazione:

```
1 // Nel file core.h
2 #define MEM_SIZE 4096           // Dimensione espressa in parole
   da 32 bit
3
4 // Nel file elf.h
5 #define RAM_SIZE 16384          // Dimensione espressa in byte
   (4096 * 4)
```

Ogni nucleo isola la propria memoria locale dichiarando il campo `uint32_t memory[MEM_SIZE]` all'interno della propria struttura di stato. Questa impostazione risponde a due precisi requisiti logici:

1. **Ottimizzazione del ciclo di Fetch:** poiché le istruzioni RV32I hanno una lunghezza fissa di 32 bit e devono essere allineate alla parola, la modellazione ad array di `uint32_t` permette di azzerare l'overhead di ricomposizione dei byte a runtime. L'accesso avviene tramite l'operazione di indicizzazione diretta `memory[pc / 4]`.
2. **Incapsulamento dello stato hardware:** L'allocazione dell'array all'interno della struct del core impedisce la sovrapposizione degli spazi di memoria. Questo vincolo software garantisce il rispetto del modello computazionale sistolico

puramente distribuito, escludendo l'esistenza (e la possibilità d'uso accidentale) di qualsiasi risorsa di memoria globale condivisa.

La macro `RAM_SIZE` interviene nel modulo ELF per gestire la coerenza di allineamento: poichè il loader opera su flussi di byte estratti dal disco, un cast temporaneo a puntatore a byte (`uint8_t*`) consente di mappare i dati reali nelle parole da 32 bit senza violare i confini fisici dell'array host.

4.1.3 Parsing e caricamento dei binari ELF

Il caricamento di un programma reale all'interno del simulatore richiede un'operazione di traduzione strutturale. Un file in formato ELF32 (Executable and Linkable Format) non si presenta come una proiezione lineare e speculare della memoria RAM, ma come un contenitore organizzato tramite tabelle descrittive e sezioni logiche distinte. Il sottosistema di caricamento gestisce questo processo attraverso due fasi sequenziali, progettate per garantire l'isolamento architetturale e ottimizzare gli accessi alle risorse dell'host.

La prima fase realizza il disaccoppiamento dal file system dell'host. La funzione `load_elf` acquisisce il file binario dal disco e ne trasferisce il contenuto grezzo in un buffer temporaneo allocato nella memoria volatile della macchina host. Questo passaggio consente di evitare accessi ripetuti e costosi al disco durante la successiva fase di decodifica, permettendo al simulatore di analizzare i metadati dell'eseguibile direttamente in memoria, con latenze ridotte.

La seconda fase esegue il parsing strutturato e la scomposizione del firmware tramite la funzione `carica_elf_in_core`. Il simulatore interpreta l'intestazione principale dell'eseguibile (`Elf32_Ehdr`) per localizzare, tramite il campo `e_shoff`, la Tabella delle Intestazioni di Sezione (`Elf32_Shdr`). Questa tabella contiene i descrittori dei vari segmenti compilati e consente al caricatore di selezionare solo le componenti necessarie all'esecuzione sul core, seguendo una logica di filtraggio basata sui metadati del binario.

```
1 // Estratto logico del parsing delle sezioni ELF
2 for (int i = 0; i < header->e_shnum; i++) {
3     if (sections[i].sh_type == SHT_PROGBITS && (sections[i].
4         sh_flags & SHF_ALLOC)) {
5         uint8_t* dest = (uint8_t*)core->memory + sections[i].
6         sh_addr;
7         memcpy(dest, elf_content + sections[i].sh_offset,
8             sections[i].sh_size);
9     }
10 }
```

La logica esecutiva di questo processo implementa un filtro deterministico per ricostruire lo stato di memoria del core senza introdurre artefatti software. L'analisi del tipo di sezione seleziona esclusivamente le aree codificate come `SHT_PROGBITS`.

Questo criterio esclude dall'allocazione i blocchi informativi destinati unicamente agli strumenti di debug dell'host, come le tabelle dei simboli (`.symtab`), garantendo che lo spazio di indirizzamento della CPU contenga soltanto istruzioni eseguibili e costanti effettivamente utilizzate.

Questa selezione viene ulteriormente affinata dal vincolo di allocazione legato al flag `SHF_ALLOC`. Questa maschera identifica le sezioni che la catena di compilazione ha destinato alla memoria durante l'esecuzione, come la sezione `.text` per il codice macchina e la sezione `.data` per le variabili statiche inizializzate. La combinazione di questi due filtri permette di trasferire i dati estratti dal buffer dell'host direttamente nell'array `memory` del core, mantenendo coerenti gli indirizzi definiti in fase di linking.

Prima della copia dei dati, il sistema verifica i limiti della memoria simulata. Il controllo assicura che la fine di ciascuna sezione non superi la dimensione disponibile della RAM, evitando così scritture fuori dai confini e possibili condizioni di overflow sul processo host.

Infine, il caricatore estrae il campo `e_entry` dall'header ELF per inizializzare il Program Counter del core. Questa operazione riproduce il comportamento di un segnale hardware di reset, impostando l'indirizzo di ingresso del programma e predisponendo il sistema all'avvio dell'esecuzione da parte del modulo che controlla il cluster.

4.2 Rappresentazione dello Stato del core

Nel paradigma della simulazione a livello di ciclo, la CPU non viene modellata a partire dalla sua implementazione fisica, ma viene rappresentata come una macchina a stati finiti. In questo modello, il core è descritto attraverso un insieme di informazioni che ne definiscono completamente il comportamento in un determinato istante.

4.2.1 Struttura dati centrale del core

Tutte le componenti dello stato architetturale di un core RV32I, insieme alle relative interfacce di input/output accoppiate, sono incapsulate nella struttura dati `RISCV_Core`:

```

1 typedef struct {
2     int         core_id;           // Identita del nodo
3     uint32_t    regs[32];          // Registri architetturali(x0-
    x31)
4     uint32_t    pc;                // Program Counter
5     uint32_t    current_inst;      // Instruction Register (IR)
6     uint32_t    memory[MEM_SIZE]; // Stato della memoria locale
7     bool        running;           // Flag di controllo
8     uint32_t    out_reg[4];        // Registri per l'output
    cardinale

```

```

9     uint32_t in_reg[4];           // Registri per l'input
    cardinale
10 } RISC_V_Core;

```

L'incapsulamento di queste variabili consente di rappresentare in modo completo lo stato interno di un singolo core, garantendone l'indipendenza logica rispetto agli altri nodi del sistema. Poiché l'intero ambiente esecutivo è contenuto all'interno della struttura, il modulo centrale può gestire l'esecuzione dei diversi core in modo sequenziale, alternandone l'avanzamento ciclo per ciclo senza introdurre interferenze tra stati distinti o fenomeni di corruzione dei dati.

4.2.2 Simulazione dei registri architetturali

Il Register File viene emulato tramite l'array lineare `uint32_t regs[32]`. Nell'ISA RISC-V, la gestione del registro `x0` rappresenta un caso di studio fondamentale per quanto riguarda la simulazione dei vincoli hardware immutabili. La regola impone che `x0` deve comportarsi come una costante fissa a zero (hardwired to ground).

In un linguaggio ad alto livello come il C, un elemento di un array è per definizione una locazione di memoria scrivibile. Per conciliare questa natura software con il vincolo hardware senza introdurre pesanti controlli condizionali all'interno di ogni singola istruzione aritmetica, è stata adottata una strategia di forzatura post-esecutiva:

```

1 void execute(RISC_V_Core *core, DecodedInstr d) {
2     // ... Logica esecutiva dell'ISA ...
3     core->regs[0] = 0; // Imposizione del vincolo hardware al
    termine del ciclo
4 }

```

Questa scelta progettuale ottimizza le prestazioni del simulatore poichè si permette temporaneamente alle istruzioni di scrivere nell'indice 0, ma si ripristina la coerenza architetturale azzerando il registro in modo deterministico subito prima che qualunque istruzione successiva possa leggerne il contenuto.

4.2.3 Canali cardinali e comunicazione tra core

La rete di interconnessione sistolica richiede che i dati si propaghino nello spazio in base a coordinate direzionali. Il simulatore mappa questa funzionalità hardware inserendo nello stato della CPU i registri di frontiera `out_reg[4]` e `in_reg[4]`. L'associazione semantica tra gli indici numerici e la topologia bidimensionale è governata da macro costanti:

```

1 #define NORD 0; #define SUD 1; #define EST 2; #define OVEST 3;

```

Questi vettori non rappresentano normali registri di calcolo, ma agiscono come l'interfaccia di frontiera del core verso l'esterno. Essi emulano il comportamento di buffer latch di rete punto-a-punto: le modifiche apportate da un core alle proprie uscite rimangono confinate nel proprio spazio logico finché l'infrastruttura del cluster non interviene per propagarle verso i nodi adiacenti.

4.2.4 Inizializzazione dello stato del core

Il passaggio da uno stato non inizializzato, conseguente all'allocazione della memoria sull'host, a uno stato coerente e deterministico del sistema è gestito dalla funzione `init_core`. Tale procedura ha il compito di riportare il core in una configurazione valida di partenza, definendo esplicitamente i parametri fondamentali di esecuzione e garantendo la consistenza dello stato architetturale prima dell'avvio del ciclo di simulazione.

```
1 void init_core(RISCV_Core *core, uint32_t start_pc, int id) {  
2     // ... azzeramento regs e memory ...  
3     for (int i = 0; i < 4; i++) {  
4         core->in_reg[i] = 0xFFFFFFFF;  
5         core->out_reg[i] = 0xFFFFFFFF;  
6     }  
7     core->pc = start_pc;  
8     core->running = true;  
9     core->core_id = id;  
10 }
```

L'inizializzazione dei registri cardinali al pattern di bit `0xFFFFFFFF` rappresenta una precisa scelta di modellazione del segnale. Poiché il valore 0 costituisce un dato numerico perfettamente valido all'interno degli algoritmi di ordinamento, usarlo come valore iniziale avrebbe impedito di distinguere un canale vuoto da un canale contenente il numero zero. Il valore `0xFFFFFFFF` assume quindi il ruolo logico di indicatore di canale non attivo o dato non disponibile (sentinel value), prevenendo l'elaborazione di input spuri nelle fasi iniziali del cluster.

4.3 Ciclo di Esecuzione del Singolo Core

L'esecuzione di un programma all'interno del simulatore è gestita dalla funzione `execute_step`, che rappresenta il meccanismo principale attraverso cui un core avanza nel proprio flusso di istruzioni. Ad ogni chiamata, la funzione esegue un singolo ciclo della CPU, aggiornandone lo stato interno in modo coerente con il comportamento previsto dall'architettura RISC-V.

Per riprodurre correttamente il funzionamento di un processore reale, l'esecuzione viene suddivisa in una sequenza di fasi distinte. Ciascuna fase svolge un compito specifico, dalla lettura dell'istruzione corrente fino all'aggiornamento dei registri e

del Program Counter. Questa organizzazione consente di modellare in modo chiaro e deterministico gli effetti prodotti da ogni istruzione sullo stato del core, garantendo che le modifiche vengano applicate nell'ordine corretto durante l'avanzamento della simulazione.

4.3.1 Fase di fetch

La prima fase del ciclo di esecuzione è il *fetch*, ovvero il prelievo dell'istruzione che dovrà essere eseguita dal core. In questa fase il core utilizza il valore corrente del Program Counter (**pc**) per individuare la posizione dell'istruzione all'interno della propria memoria locale e copiarla nel registro **current_inst**, che svolge il ruolo di registro istruzione.

```
1 void execute_step(RISCV_Core *core) {
2     if (!core->running) return;
3     if ((core->pc / 4) >= MEM_SIZE) { core->running = false;
        return; } // Termina se il PC supera i limiti della memoria
        simulata
4 core->running = false; return ; }
5
6     core->current_inst = core->memory[core->pc / 4];
7     core->pc += 4; // Avanzamento sequenziale
8
9     DecodedInstr d = decode(core->current_inst);
10    execute(core, d);
11 }
```

Prima di effettuare il prelievo, il simulatore verifica che il valore del Program Counter rientri nei limiti della memoria disponibile. Se l'indirizzo risulta esterno allo spazio di memoria del core, l'esecuzione viene terminata impostando il flag **running** a **false**. Questo controllo impedisce accessi non validi alla memoria simulata e garantisce la correttezza dell'esecuzione.

Una volta recuperata l'istruzione, il Program Counter viene incrementato di 4 byte. Tale incremento riflette la dimensione fissa delle istruzioni dell'ISA RV32I, pari a una word da 32 bit. In condizioni normali, infatti, l'istruzione successiva si trova immediatamente dopo quella corrente.

L'aggiornamento del Program Counter avviene prima della fase di esecuzione, assumendo temporaneamente che il flusso del programma prosegua in modo sequenziale. Nel caso in cui l'istruzione corrente sia un salto o una diramazione condizionata, il valore del **pc** verrà successivamente corretto dalla logica di esecuzione. Questo approccio riproduce il comportamento adottato da molti processori reali e semplifica l'organizzazione del ciclo macchina all'interno del simulatore.

4.3.2 Fase di decode

La fase di decodifica ha il compito di interpretare l'istruzione appena prelevata dalla memoria e di estrarne tutte le informazioni necessarie all'esecuzione. Per semplificare le fasi successive del ciclo macchina, il simulatore memorizza tali informazioni nella struttura `DecodedInstr`, che rappresenta una descrizione già elaborata dell'istruzione.

L'architettura RISC-V è stata progettata per facilitare la decodifica hardware. I campi che identificano i registri sorgente (`rs1` e `rs2`) e il registro destinazione (`rd`) occupano infatti posizioni fisse all'interno della parola a 32 bit. Questo permette al decoder di estrarli direttamente tramite maschere e operazioni di shift, indipendentemente dal tipo di istruzione considerato.

La parte più complessa del processo riguarda invece la ricostruzione dei valori immediati. A differenza dei registri, i bit che compongono un immediato non sono sempre memorizzati in posizioni contigue. La loro disposizione varia in funzione del formato dell'istruzione e richiede quindi una fase di ricomposizione prima che il valore possa essere utilizzato dall'unità di esecuzione.

```

1 DecodedInstr decode(uint32_t inst) {
2     DecodedInstr d;
3     // ... estrazione campi fissi rs1, rs2, rd, funct3, funct7
4     ...
5     d.opcode = inst & 0x7F;
6
7     switch (d.opcode) {
8         case OP_IMM: // Formato I-Type (Aritmetica immediata)
9             d.imm = (((int32_t)inst) >> 20);
10            break;
11        case STORE: // Formato S-Type (Memorizzazione in RAM)
12            d.imm = (((int32_t)inst) >> 25) << 5 | (((inst
13            >> 7) & 0x1F));
14            if (d.imm & 0x800) d.imm |= 0xFFFFF000; // Sign-
15            extension manuale
16            break;
17        case BRANCH: // Formato B-Type (Salti condizionati)
18            d.imm = (((inst >> 31) & 0x1) << 12) | (((inst >>
19            7) & 0x1) << 11) |
20            (((inst >> 25) & 0x3F) << 5) | (((inst >>
21            8) & 0xF) << 1);
22            if (d.imm & 0x1000) d.imm |= 0xFFFFE000;
23            break;
24        // ... formato U-Type e J-Type ...
25    }
26    return d;
27 }
```

Un aspetto fondamentale di questa fase è l'estensione del segno (*sign-extension*). Molti immediati dell'ISA RV32I occupano meno di 32 bit e possono rappresentare

sia valori positivi sia valori negativi. Prima di essere utilizzati nelle operazioni aritmetiche, tali valori devono quindi essere convertiti in interi a 32 bit mantenendo inalterato il loro significato numerico.

Nel formato I-Type, l'immediato a 12 bit viene estratto dai bit 31:20 della parola istruzione tramite uno shift logico a destra di 20 posizioni, isolando i bit rilevanti nel campo inferiore. L'estensione del segno viene poi eseguita sfruttando il comportamento degli shift aritmetici del linguaggio C: il valore estratto viene prima promosso a intero con segno tramite cast (`int32_t`), quindi traslato a sinistra di 20 posizioni e immediatamente ritraslato a destra dello stesso numero, ottenendo la propagazione del bit di segno su tutti i bit superiori. Questa tecnica, corrispondente all'espressione `(int32_t)(imm_i << 20) >> 20`, è equivalente a un'estensione manuale tramite maschera ma risulta più compatta e sfrutta direttamente le garanzie dello shift aritmetico su tipi con segno in C.

Nei formati S-Type e B-Type, dove i bit dell'immediato sono distribuiti in posizioni non contigue, la ricomposizione avviene combinando i frammenti con operazioni logiche, dopodichè si applica la medesima tecnica di estensione manuale del segno tramite maschera `0xFFFFF000` o `0xFFFFE000`.

4.3.3 Fase di execute

La fase di esecuzione è responsabile dell'applicazione concreta delle operazioni richieste dall'istruzione decodificata. In questa fase il simulatore modifica lo stato del core in funzione dell'opcode e dei relativi campi di controllo, riproducendo il comportamento previsto dall'ISA RISC-V. La logica è implementata attraverso una struttura `switch-case`, nella quale ogni opcode viene associato alla corrispondente operazione architetturale.

```

1 void execute(RISCV_Core *core, DecodedInstr d) {
2     uint32_t val1 = core->regs[d.rs1];
3     uint32_t val2 = core->regs[d.rs2];
4
5     switch (d.opcode) {
6         case OP_IMM: // ADDI
7             if (d.funct3 == 0x0) core->regs[d.rd] = val1 + d.
            imm;
8             break;
9         case OP: // ADD
10            if (d.funct3 == 0x0 && d.funct7 == 0x00) core->
            regs[d.rd] = val1 + val2;
11            break;
12        case LOAD: // LW (Load Word)
13            if (d.funct3 == 0x2) core->regs[d.rd] = core->
            memory[(val1 + d.imm) / 4];
14            break;
15        case STORE: // SW (Store Word)

```

```

16         if (d.funct3 == 0x2) core->memory[(val1 + d.imm) /
17         4] = val2;
18         break;
19         case BRANCH: // Salti condizionati (es. BEQ)
20         if (d.funct3 == 0x0 && val1 == val2) core->pc = (
21         core->pc - 4) + d.imm;
22         break;
23         case JAL: // Salto incondizionato diretto
24         core->regs[d.rd] = core->pc;
25         core->pc = (core->pc - 4) + d.imm;
26         break;
27     }
    core->regs[0] = 0;
}

```

L'analisi del codice mostra come le diverse categorie di istruzioni producano effetti differenti sullo stato del core.

Le istruzioni aritmetiche e logiche, come ADD e ADDI, operano esclusivamente sui registri generali. In questi casi il simulatore legge gli operandi dal Register File, esegue il calcolo richiesto e memorizza il risultato nel registro di destinazione.

Le istruzioni di accesso alla memoria, come LW e SW, consentono invece di trasferire dati tra i registri e la memoria locale del core. L'indirizzo effettivo viene ottenuto sommando il contenuto di un registro base al valore immediato dell'istruzione. Poiché la memoria simulata è rappresentata come un array di parole da 32 bit, l'indirizzo calcolato viene diviso per quattro prima di accedere all'elemento corrispondente.

Le istruzioni di controllo del flusso, come BEQ e JAL, modificano direttamente il Program Counter e consentono di alterare la sequenza normale di esecuzione. In questi casi il simulatore deve tenere conto del fatto che il pc è già stato incrementato durante la fase di fetch. Per questo motivo il nuovo indirizzo viene calcolato utilizzando l'espressione `(core->pc - 4) + d.imm`, che riporta temporaneamente il Program Counter all'indirizzo dell'istruzione corrente prima di applicare l'offset di salto.

Al termine di ogni esecuzione viene inoltre ripristinato il valore del registro x0 mediante l'istruzione `core->regs[0] = 0`. Questa operazione garantisce il rispetto di uno dei vincoli fondamentali dell'architettura RISC-V come già illustrato nei paragrafi precedenti.

4.4 Modello Multi-Core Sincrono e Propagazione Sistolica

Il passaggio dalla simulazione di un singolo core a quella di un cluster composto da più processori introduce nuove problematiche legate alla sincronizzazione dell'esecuzione. Sebbene il sistema modellato preveda più nodi che operano in modo coordinato, il simulatore viene eseguito su una macchina host che elabora le istru-

zioni in modo sequenziale. Diventa quindi necessario adottare un meccanismo che permetta di rappresentare correttamente l'avanzamento simultaneo dei diversi core, preservando la coerenza delle comunicazioni e dello stato del sistema a ogni ciclo di clock.

4.4.1 Problema della concorrenza in simulazione sequenziale

In un'architettura hardware reale, tutti i core sono sincronizzati da un segnale di clock comune. A ogni fronte di clock, ciascun core esegue le proprie operazioni e aggiorna il proprio stato in modo simultaneo rispetto agli altri.

In una simulazione eseguita su una macchina host, questo comportamento deve essere approssimato tramite esecuzione sequenziale. Tuttavia, un aggiornamento diretto dei core uno alla volta (ad esempio Core 0, poi Core 1, e così via) introduce un problema di coerenza temporale. In questo caso, un core potrebbe leggere valori già aggiornati da un altro core nello stesso ciclo logico, alterando la corretta evoluzione del sistema.

Questo fenomeno rompe il modello sistolico originale, in cui ogni dato deve avanzare di un solo passo per ciclo di clock. Per evitare questo problema è necessario adottare un meccanismo di simulazione che separi la fase di lettura dello stato dalla fase di aggiornamento, garantendo che tutti i core osservino lo stesso stato iniziale all'interno dello stesso ciclo.

4.4.2 Snapshot degli ingressi

Per risolvere questa sfasatura temporale senza introdurre costrutti di multithreading sul sistema host, che renderebbero il comportamento dipendente dallo scheduling del sistema operativo, nel modulo `main.c` è stato adottato un approccio basato su doppia bufferizzazione logica.

Il ciclo globale di clock gestito dal modulo centrale viene quindi suddiviso in fasi separate e ben definite. La prima fase consiste nel congelamento dello stato dei canali di comunicazione tramite la creazione di uno snapshot degli ingressi:

```
1 // All'interno della funzione run_multi_core nel file main.c
2 uint32_t snap_in[N][4];
3
4 // Fase 1: Isolamento temporale - Congelamento degli ingressi
5 for (int i = 0; i < n; i++) {
6     for (int d = 0; d < 4; d++) {
7         snap_in[i][d] = cluster[i].in_reg[d];
8     }
9 }
```

La matrice `snap_in` rappresenta uno stato temporaneo del sistema e ha la funzione di garantire la sincronizzazione tra i core. In questo modo, tutti i processori

lavorano sullo stesso insieme di valori di ingresso relativi all'inizio del ciclo di clock, evitando che aggiornamenti intermedi possano influenzare l'esecuzione corrente.

4.4.3 Esecuzione isolata dei core

Nella seconda fase, il simulatore procede con l'aggiornamento dei singoli nodi del cluster, garantendo che ciascun core operi in uno stato di isolamento logico rispetto agli altri. Ogni core esegue la funzione `execute_step` utilizzando esclusivamente i dati provenienti dallo snapshot degli ingressi.

```

1 // Fase 2: Avanzamento di stato in isolamento condizionato
2 for (int i = 0; i < n; i++) {
3     if (cluster[i].running) {
4         for (int d = 0; d < 4; d++) {
5             cluster[i].in_reg[d] = snap_in[i][d]; // Forzatura
6             del vincolo temporale
7         }
8         execute_step(&cluster[i]);
9     }

```

Questo approccio consente di ottenere una forma di concorrenza logica indipendente dall'ordine di esecuzione sul sistema host. Poiché ogni core legge esclusivamente i propri ingressi dalla matrice `snap_in`, l'ordine con cui i nodi vengono aggiornati (ad esempio Core 0 prima del Core 3 o viceversa) non influisce sul risultato della simulazione.

Eventuali modifiche prodotte da un core nei propri registri di uscita `out_reg` vengono registrate nello stato del cluster, ma non sono immediatamente visibili agli altri nodi. Tali aggiornamenti diventeranno effettivi solo nel ciclo di simulazione successivo, preservando così la coerenza temporale del modello sistolico.

4.4.4 Propagazione tra core e condizioni al contorno

Una volta completato l'avanzamento isolato di tutti i core, si entra nella terza e ultima fase del ciclo di clock globale: la propagazione dei segnali lungo la topologia lineare del cluster. In questa fase il simulatore trasferisce i valori depositati nei registri di uscita di ciascun core verso i registri di ingresso del nodo adiacente, realizzando il trasporto fisico dei dati lungo la catena sistolica.

```

1 // Fase 3: Propagazione spaziale lungo la topologia lineare
2 for (int i = 0; i < n; i++) {
3     if (i < n - 1) cluster[i + 1].in_reg[OVEST] = cluster[i].
4     out_reg[EST];
5     if (i > 0) cluster[i - 1].in_reg[EST] = cluster[i].
6     out_reg[OVEST];
7 }

```

```

6
7 // Gestione dinamica dei confini esterni
8 if (step < MAX_STEPS) {
9     cluster[0].in_reg[OVEST] = inOvest_vettore[step];
10    cluster[n - 1].in_reg[EST] = inEst_vettore[step];
11 }

```

Si noti che nella topologia lineare implementata la propagazione è limitata ai soli canali Est e Ovest. I registri Nord e Sud sono inizializzati al valore sentinella `0xFFFFFFFF` e non vengono aggiornati dal modulo di propagazione, restando disponibili per estensioni future verso reti bidimensionali. Questa operazione è distinta e successiva alla fase di calcolo: separare propagazione ed esecuzione garantisce che nessun core possa osservare dati prodotti dai vicini nello stesso ciclo in cui vengono generati.

4.5 Estensione dell'ISA: Istruzione Custom 0x0B

Per consentire ai programmi assemblati di interagire in modo diretto con l'infrastruttura di rete sistolica, è stata introdotta un'estensione dell'Instruction Set Architecture (ISA) del core, aggiungendo il supporto esplicito a primitive di input/output cardinale.

4.5.1 Integrazione dell'opcode personalizzato

Nei sistemi embedded tradizionali, l'interazione con le periferiche avviene tramite Memory-Mapped I/O (MMIO), in cui i dispositivi sono associati a specifici indirizzi di memoria. Questo approccio, sebbene semplice, introduce nel caso dei sistemi sistolici un overhead aggiuntivo, poiché ogni operazione di I/O richiede istruzioni di accesso alla memoria e calcolo degli indirizzi.

Per ridurre questa inefficienza, lo spazio degli opcode del decoder è stato esteso introducendo l'opcode personalizzato `0x0B`, appartenente alla categoria delle estensioni custom dello standard RISC-V. Questa scelta mantiene la compatibilità con l'ISA base RV32I, poiché tale opcode è riservato e non confligge con istruzioni standard, permettendo l'introduzione di primitive dedicate alla comunicazione tra nodi.

4.5.2 Istruzione IN

L'istruzione IN permette il trasferimento di un dato da un canale cardinale di ingresso al Register File, senza passare attraverso la memoria principale. L'istruzione è identificata dal campo `funct3 == 0x0`, mentre il campo `rs2` viene reinterpretato come identificatore della direzione (0–3).

```

1 // All'interno dello switch(d.opcode) nella funzione execute
   in core.c

```

```
2 case 0x0B: {  
3     int dir = d.rs2; // Reinterpretazione semantica del campo  
   rs2 come direzione  
4     if (d.funct3 == 0x0) { // Primitiva logica IN  
5         core->regs[d.rd] = core->in_reg[dir];  
6         printf("IN x%d, DIR:%d\n", d.rd, dir);  
7     }
```

Questa istruzione modella una lettura diretta da un canale di ingresso: il valore presente in `in_reg[dir]` viene trasferito nel registro di destinazione `regs[d.rd]` in un'unica operazione.

4.5.3 Istruzione OUT

L'istruzione OUT realizza l'operazione inversa, trasferendo il contenuto di un registro verso il canale di uscita corrispondente. Come per l'istruzione IN, la direzione cardinale di destinazione è codificata nel campo `rs2` dell'istruzione, mentre il dato da trasmettere è letto dal registro sorgente `rs1`. L'istruzione viene identificata dal campo `funct3 == 0x1`.

```
1 else if (d.funct3 == 0x1) { // Primitiva logica OUT  
2     core->out_reg[dir] = core->regs[d.rs1];  
3     printf("OUT DIR:%d, x%d\n", dir, d.rs1);  
4 }  
5 } break;
```

In questo caso, il valore contenuto in `regs[d.rs1]` viene scritto nel registro di uscita `out_reg[dir]`. Il dato rimane disponibile fino alla fase di propagazione successiva, quando il modulo centrale del cluster lo trasferisce verso il nodo adiacente, realizzando così una comunicazione punto-a-punto tra core.

4.6 Strumenti di Debug e Osservazione

Lo sviluppo e la validazione di algoritmi paralleli e distribuiti, come l'ordinamento sistolico, presentano difficoltà intrinseche legate alla complessità nell'ispezione dei flussi di dati tra nodi interconnessi. Per affrontare questo problema, il simulatore integra un'infrastruttura di osservabilità progettata per monitorare in modo diretto l'evoluzione dello stato globale del cluster durante l'esecuzione.

4.6.1 Ispezione dello stato del cluster

Lo strumento principale per l'ispezione dello stato interno di ciascun core è la funzione `print_state`, definita nel modulo `core.c` e richiamata dal modulo centrale al termine di ogni ciclo di clock simulato.

```

1 void print_state(RISCV_Core *core) {
2     printf("\n--- STATO CORE [%d] | PC: 0x%08x | INSTR: 0x%08x
3     ---\n", core->core_id, core->pc, core->current_inst);
4     // Stampa del Register File organizzato in righe da 4
5     colonne
6     for (int i = 0; i < 32; i++) {
7         printf("  x%02d= %-10d", i, (int32_t)core->regs[i]);
8         if ((i + 1) % 4 == 0) printf("\n");
9     }
10    // Monitoraggio simultaneo della frontiera cardinale di I/
11    0
12    printf("  IN : NORD=%-6d SUD=%-6d EST=%-6d OVEST=%-6d\n",
13    (int32_t)core->in_reg[0], (int32_t)core->in_reg[1], (
14    int32_t)core->in_reg[2], (int32_t)core->in_reg[3]);
15    printf("  OUT: NORD=%-6d SUD=%-6d EST=%-6d OVEST=%-6d\n",
16    (int32_t)core->out_reg[0], (int32_t)core->out_reg[1], (
17    int32_t)core->out_reg[2], (int32_t)core->out_reg[3]);
18 }

```

La visualizzazione congiunta dei registri interni e delle interfacce di frontiera (`in_reg` e `out_reg`) consente una tracciabilità completa dello stato del core. In questo modo è possibile verificare in modo diretto il corretto funzionamento del meccanismo di isolamento temporale, assicurando che i dati prodotti da un nodo non influenzino gli ingressi degli altri core nello stesso ciclo di clock.

4.6.2 Monitoraggio dei registri di esecuzione

Oltre all'analisi del singolo core, il simulatore prevede una forma di monitoraggio globale integrata nel ciclo principale del file `main.c`. Questa routine effettua un campionamento mirato di alcune variabili significative per l'analisi del comportamento dell'algoritmo di ordinamento distribuito.

```

1 // Monitoraggio diagnostico selettivo nel ciclo di clock in
2 main.c
3 for (int i = 0; i < n; i++) {
4     printf("  Core [%d]: x6 = %-10d | In_Ovest = %-10d |
5     Out_Est = %-10d\n",
6     cluster[i].core_id, (int32_t)cluster[i].regs[6],
7     (int32_t)cluster[i].in_reg[OVEST], (int32_t)cluster
8     [i].out_reg[EST]);
9 }

```

Questo tipo di tracciamento selettivo è particolarmente utile per la validazione del comportamento sistolico. Il monitoraggio dell'evoluzione di registri specifici, come `x6`, permette di osservare in modo diretto la dinamica dell'algoritmo ciclo dopo ciclo.

In questo modo è possibile seguire la propagazione dei valori lungo la catena di core e verificare il corretto funzionamento dei meccanismi di confronto e scam-

bio, osservando la convergenza progressiva del sistema verso lo stato finale previsto dall'algoritmo.

Capitolo 5

Esperimenti e Validazioni

5.1 Test funzionali su singola istruzione

La prima fase di validazione è stata dedicata alla verifica del corretto funzionamento del motore di esecuzione del simulatore. L'obiettivo era accertare che le istruzioni del subset RV32I producessero le modifiche attese sullo stato del core, verificando in particolare le fasi di decodifica, gestione degli immediati, aggiornamento dei registri e controllo del flusso di esecuzione.

Per questo scopo è stato sviluppato un semplice programma assembly che calcola la sommatoria dei primi cinque numeri interi mediante un ciclo iterativo:

```
1 .global _start
2
3 _start:
4 addi x1, x0, 5
5 addi x2, x0, 0
6 addi x3, x0, 1
7
8 loop:
9 bge x3, x1, end
10 add x2, x2, x3
11 addi x3, x3, 1
12 jal x0, loop
13
14 end:
15 add x2, x2, x3
```

Questo programma consente di verificare contemporaneamente diverse classi di istruzioni. Le prime istruzioni `addi` inizializzano i registri utilizzati dall'algoritmo e permettono di controllare il corretto utilizzo degli operandi immediati. Le istruzioni `add` verificano invece il funzionamento delle operazioni aritmetiche tra registri.

Particolare interesse riveste il ciclo principale del programma, che permette di analizzare il comportamento delle istruzioni di controllo del flusso. L'istruzione `bge`

confronta il contenuto dei registri `x3` e `x1` e decide se proseguire l'esecuzione sequenziale oppure terminare il ciclo. L'istruzione `jal` aggiorna invece il Program Counter riportando l'esecuzione all'etichetta `loop`, consentendo di verificare il corretto calcolo degli offset relativi e la gestione dei salti incondizionati.

Al termine dell'esecuzione è stato analizzato lo stato finale del core tramite i dati mostrati dal simulatore:

```

--- STATO CORE [0] | PC: 0x00000020 | INSTR: 0x0000006f ---
x00(zero)= 0      x01(ra )= 5      x02(sp )= 15      x03(gp )= 5
x04(tp )= 0      x05(t0 )= 0      x06(t1 )= 0      x07(t2 )= 0
x08(s0 )= 0      x09(s1 )= 0      x10(a0 )= 0      x11(a1 )= 0
x12(a2 )= 0      x13(a3 )= 0      x14(a4 )= 0      x15(a5 )= 0
x16(a6 )= 0      x17(a7 )= 0      x18(s2 )= 0      x19(s3 )= 0
x20(s4 )= 0      x21(s5 )= 0      x22(s6 )= 0      x23(s7 )= 0
x24(s8 )= 0      x25(s9 )= 0      x26(s10)= 0      x27(s11)= 0
x28(t3 )= 0      x29(t4 )= 0      x30(t5 )= 0      x31(t6 )= 0
IN : NORD=-1      SUD=-1      EST=-1      OVEST=-1
OUT: NORD=-1      SUD=-1      EST=-1      OVEST=-1

```

Figura 5.1: Stato finale del core

I risultati ottenuti confermano il corretto funzionamento del simulatore. Il registro accumulatore `x2` contiene infatti il valore 15, corrispondente alla somma dei numeri da 1 a 5. Inoltre, il registro `x3` raggiunge il valore limite previsto e il Program Counter termina sull'istruzione finale del programma.

L'ispezione dello stato finale conferma anche il rispetto del vincolo architetturale sul registro `x0`, che mantiene il valore zero per tutta l'esecuzione. Questo test fornisce quindi una prima verifica della correttezza delle operazioni aritmetiche, delle istruzioni di salto e dell'aggiornamento dello stato interno del core.

5.2 Caso di studio: algoritmo di Parallel Sort su rete sistolica

Per verificare il corretto funzionamento della comunicazione tra i core e delle istruzioni personalizzate introdotte nel simulatore, è stato utilizzato come caso di studio un algoritmo di ordinamento parallelo basato su una rete sistolica lineare. Questo test permette di osservare il comportamento dell'intero sistema durante l'esecuzione di uno scenario realistico, in cui più nodi elaborano e trasferiscono dati in modo continuo.

L'obiettivo non è soltanto verificare la corretta esecuzione delle istruzioni `IN` e `OUT`, ma anche controllare che il meccanismo di sincronizzazione basato sugli snapshot mantenga la corretta sequenza temporale degli scambi tra i core. In una rete sistolica, infatti, ogni dato deve avanzare di un solo nodo per ciclo di clock e non deve propagarsi istantaneamente lungo l'intera catena.

L'algoritmo prevede l'inserimento di una sequenza di numeri attraverso l'ingresso `OVEST` del primo core. Ogni nodo conserva localmente il valore minimo osservato

fino a quel momento e inoltra verso est il valore più grande. Con il passare dei cicli, i valori si distribuiscono progressivamente lungo la catena fino a raggiungere una configurazione ordinata.

5.2.1 Configurazione dell'ambiente sperimentale e codice Assembly

L'esperimento è stato eseguito utilizzando un cluster lineare composto da $N = 4$ core collegati punto a punto. Tutti i processori eseguono lo stesso programma Assembly, responsabile della ricezione dei dati, del confronto con il valore locale e dell'eventuale inoltro verso il nodo successivo. Il frammento principale del firmware è riportato di seguito:

```
1 _start:
2     addi    t1, x0, -1          ; Inizializza il buffer locale (t1)
3     a      -1
4 loop:
5     IN      5, 3                ; 1. Legge il dato entrante da
6     OVEST in t0 (x5)
7     bltu    t0, t1, swap        ; 2. Se t0 < t1 (unsigned) vai a
8     swap
9     mv      t2, t0              ; 3. t0 >= t1: t2 = t0 (il piu'
10    grande, da mandare)
11    j       out                 ; 4. Salta a out
12 swap:
13    mv      t2, t1              ; 3'. t2 = vecchio buffer (il piu'
14    grande)
15    mv      t1, t0              ; 4'. t1 = t0 (nuovo minimo locale)
16 out:
17    OUT     7, 2                ; 5. Manda t2 (x7) verso EST (2)
18    j       loop                ; 6. Ricomincia
```

La logica di ogni core è strutturata in modo minimale: alla ricezione di un nuovo dato dalla porta OVEST, il core confronta il valore ricevuto `t0` con il proprio buffer locale `t1`. Se il dato ricevuto è minore, i due valori vengono scambiati: `t1` trattiene il nuovo minimo e `t2` assume il valore precedente da inoltrare verso est. In caso contrario, è il dato ricevuto stesso ad essere inoltrato senza modificare il buffer locale. In entrambi i casi, il valore in `t2` viene trasmesso verso il nodo adiacente tramite l'istruzione `OUT`, e il ciclo ricomincia immediatamente.

Vale la pena osservare che la gestione del valore sentinella non richiede un ramo dedicato: poiché `bltu` opera in aritmetica unsigned e il valore `-1` corrisponde a `0xFFFFFFFF`, esso risulta automaticamente il massimo rappresentabile. Di conseguenza, qualsiasi dato reale in ingresso sarà sempre minore del valore iniziale di `t1`,

e verrà memorizzato nel buffer locale al primo ciclo utile senza necessità di logica aggiuntiva.

Per la validazione è stato utilizzato il vettore di ingresso:

$$V_{in} = [45, 12, 89, 7]$$

I valori vengono immessi uno alla volta nell'ingresso occidentale del primo core e attraversano progressivamente la rete. Grazie al comportamento locale di confronto e scambio eseguito da ogni nodo, il sistema converge gradualmente verso una distribuzione ordinata dei dati, permettendo di verificare sia la correttezza dell'algoritmo sia il corretto funzionamento della comunicazione tra i processori.

5.2.2 Analisi dei log di esecuzione

Uno degli strumenti più utili per verificare il comportamento del simulatore è il sistema di tracciamento integrato, che permette di osservare l'evoluzione dello stato del cluster ciclo dopo ciclo. Attraverso i log è possibile controllare sia il contenuto dei registri interni dei core sia lo stato dei canali di comunicazione utilizzati per lo scambio dei dati.

Durante l'esecuzione dell'algoritmo di ordinamento, il simulatore mostra per ogni ciclo di clock le istruzioni eseguite dai diversi core, il contenuto delle interfacce di ingresso e uscita e il valore memorizzato localmente da ciascun nodo. In questo modo è possibile seguire visivamente il percorso dei dati mentre attraversano la rete sistolica.

Nel ciclo iniziale tutti i core eseguono la stessa istruzione di inizializzazione, impostando il proprio deposito locale al valore sentinella -1. Nello stesso istante il primo elemento della sequenza da ordinare viene presentato all'ingresso occidentale del Core 0. Tutti gli altri canali risultano ancora vuoti, come si può notare dalla [Figura 5.2](#).

```
CICLO DI CLOCK 0
```

```
ADDI x6, x0, -1
ADDI x6, x0, -1
ADDI x6, x0, -1
ADDI x6, x0, -1
```

Stato interfacce (dopo Fase B):

Core	IN_OV	OUT_OV	IN_EST	OUT_EST
Core [0]	45	-1	-1	-1
Core [1]	-1	-1	-1	-1
Core [2]	-1	-1	-1	-1
Core [3]	-1	-1	-1	-1

Valori nel deposito di ogni core:

```
Core[0]: -1 | Core[1]: -1 | Core[2]: -1 | Core[3]: -1
```

Figura 5.2: Ciclo di Clock 0

Analizzando i cicli successivi si osserva il funzionamento del meccanismo di comunicazione tra i nodi. Quando un core esegue un'istruzione `OUT`, il valore viene scritto nel proprio registro di uscita ma non viene immediatamente utilizzato dagli altri processori. Il trasferimento effettivo avviene solo nella fase di propagazione successiva, quando il dato viene copiato nell'ingresso del nodo adiacente. Questo comportamento conferma il corretto funzionamento del modello a snapshot descritto nel capitolo precedente e garantisce che ogni dato avanzi di un solo nodo per ciclo di clock.

Un esempio significativo è rappresentato dal trasferimento del valore 45 dal Core 0 al Core 1. Dopo il confronto con il valore successivo, il Core 0 trattiene localmente il numero 12 e inoltra verso est il valore 45, quello che accade è rappresentato dalla [Figura 5.3](#). Nel ciclo seguente il Core 1 riceve il dato attraverso il proprio ingresso occidentale e può elaborarlo secondo la stessa logica di confronto e scambio.

```
CICLO DI CLOCK 11
```

```
OUT x7, DIR:2 (valore: 45)
OUT x7, DIR:2 (valore: -1)
OUT x7, DIR:2 (valore: -1)
OUT x7, DIR:2 (valore: -1)
```

Stato interfacce (dopo Fase B):

Core	IN_OV	OUT_OV	IN_EST	OUT_EST
Core [0]	-1	-1	-1	45
Core [1]	45	-1	-1	-1
Core [2]	-1	-1	-1	-1
Core [3]	-1	-1	-1	-1

Valori nel deposito di ogni core:

```
Core[0]: 12 | Core[1]: -1 | Core[2]: -1 | Core[3]: -1
```

Figura 5.3: Ciclo di Clock 11

Al termine dell'esecuzione, i log mostrano che i valori risultano distribuiti nei depositi locali dei quattro nodi nell'ordine:

[7, ; 12, ; 45, ; 89]

Il risultato conferma il corretto funzionamento dell'algoritmo di ordinamento distribuito. Inoltre, durante tutta l'esecuzione, i canali non utilizzati mantengono il valore sentinella -1, senza mostrare propagazioni indesiderate o interferenze tra nodi adiacenti. Questo comportamento costituisce un'ulteriore verifica della correttezza del meccanismo di comunicazione implementato nel simulatore.

5.2.3 Analisi empirica dei tempi di convergenza

La validazione funzionale e prestazionale dell'array sistolico è stata effettuata misurando il numero di cicli di clock necessari al cluster per completare l'ordinamento e raggiungere uno stato stabile. Sulla base della traccia di esecuzione analizzata,

CICLO DI CLOCK 40

```
JAL x0, 12
JAL x0, 12
JAL x0, 12
ADDI x6, x5, 0
```

Stato interfacce (dopo Fase B):

Core	IN_OV	OUT_OV	IN_EST	OUT_EST
Core [0]	-1	-1	-1	-1
Core [1]	-1	-1	-1	-1
Core [2]	-1	-1	-1	89
Core [3]	89	-1	-1	-1

Valori nel deposito di ogni core:

Core[0]: 7 Core[1]: 12 Core[2]: 45 Core[3]: 89

Figura 5.4: Ciclo di Clock 40

l'algoritmo converge in modo deterministico al ciclo di clock 40. Questo risultato è coerente con il comportamento atteso di un array sistolico lineare per l'ordinamento: per un cluster di N core che elabora una sequenza di N elementi, il numero di cicli necessari alla convergenza scala linearmente con N , in accordo con la complessità temporale $O(N)$ tipica delle reti sistoliche per il bubble sort [3]. Nel caso in esame, con $N = 4$ e un passo di iniezione (*stride*) pari a 6 cicli tra un elemento e il successivo, il tempo totale di elaborazione rispecchia questa relazione. In termini più precisi, il tempo totale di convergenza può essere espresso come:

$$T = (N - 1) \cdot s + N \cdot c$$

dove N è il numero di core, s è lo stride di iniezione (numero di cicli tra un elemento e il successivo) e c è il numero di cicli di clock necessari a ciascun core per completare un'iterazione del proprio ciclo interno di confronto. Nel caso in esame, con $N = 4$, $s = 6$ e $c \approx 4$ cicli per iterazione assembly, si ottiene:

$$T \approx (4 - 1) \cdot 6 + 4 \cdot 4 = 18 + 16 = 34 \text{ cicli teorici}$$

a cui si aggiungono i cicli di latenza iniziale per il riempimento della pipeline, coerentemente con il valore osservato di 40 cicli. La dipendenza lineare da N conferma la scalabilità del modello sistolico.

In tale istante, come evidenziato nella [Figura 5.4](#), registri di deposito dei singoli core risultano ordinati lungo la catena secondo la sequenza:

$$\text{Core}[0]: 7, \quad \text{Core}[1]: 12, \quad \text{Core}[2]: 45, \quad \text{Core}[3]: 89$$

Nel ciclo successivo (ciclo 41), tutte le interfacce di comunicazione tornano al valore di riposo -1, indicando l'esaurimento del flusso di propagazione dei dati e il raggiungimento della condizione di stabilità del sistema.

I risultati ottenuti permettono di evidenziare due proprietà fondamentali del modello implementato. La prima riguarda l'assenza di condizioni di stallo: le operazioni

di ingresso e uscita risultano perfettamente sincronizzate e non emergono situazioni di blocco o inconsistenze temporali nella propagazione dei dati tra i nodi.

La seconda proprietà riguarda la prevedibilità del comportamento sistolico. Il completamento dell'ordinamento in un numero finito di cicli conferma la regolarità del modello e la sua coerenza con la teoria delle reti sistoliche. In particolare, per un cluster composto da quattro core, il sistema raggiunge la configurazione ordinata in 40 cicli complessivi, dimostrando una dinamica stabile e ripetibile.

Capitolo 6

Conclusioni e Sviluppi Futuri

6.1 Sintesi del lavoro svolto

Il lavoro presentato in questa tesi ha portato alla realizzazione di un simulatore funzionante per architetture multi-core RISC-V, con un contributo originale nella definizione di primitive di comunicazione inter-core integrate direttamente nell'ISA. A differenza degli approcci basati su memoria condivisa, il modello proposto trasforma lo scambio di dati tra processori in operazioni su registri locali, eliminando le problematiche di contesa tipiche dei sistemi a memoria comune e rendendo il comportamento del cluster deterministico e analiticamente prevedibile. Il caso di studio sul Parallel Sort ha permesso di verificare non solo la correttezza funzionale delle singole componenti, ma anche la coerenza globale del modello di sincronizzazione: i dati avanzano di un nodo per ciclo, la convergenza avviene in un numero di passi proporzionale a NN , e l'isolamento temporale tra fase di lettura e fase di aggiornamento garantisce l'assenza di race condition nella simulazione sequenziale. Il risultato principale non è quindi soltanto il simulatore in sé, ma la dimostrazione che un modello sistolico con comunicazione diretta tra core può essere implementato in modo compatto, verificabile e facilmente estendibile, a partire da strumenti standard come il linguaggio C e l'architettura aperta RISC-V.

6.2 Considerazioni sull'architettura implementata

L'architettura software sviluppata presenta alcune caratteristiche chiave che emergono direttamente dalla sua struttura implementativa.

Un primo aspetto riguarda la separazione del ciclo di simulazione in fasi distinte, che consente di distinguere chiaramente tra fase di esecuzione locale del singolo core e fase di aggiornamento delle interfacce di comunicazione. Questo approccio elimina la possibilità che aggiornamenti prodotti all'interno di un ciclo di clock influenzino immediatamente altri core, garantendo un comportamento deterministico del sistema.

Un secondo elemento riguarda la gestione dei canali di comunicazione. L'utilizzo di registri di frontiera per l'ingresso e l'uscita consente di modellare la comunicazione inter-core come scambio esplicito di dati tra strutture isolate, mantenendo invariata la consistenza dello stato interno di ciascun core durante l'esecuzione del ciclo.

Infine, l'implementazione in linguaggio C garantisce un controllo diretto sulle strutture dati e sulle operazioni a basso livello, rendendo possibile una traduzione abbastanza fedele dei concetti hardware in un modello software semplificato ma coerente.

È tuttavia opportuno evidenziare alcuni limiti strutturali dell'architettura implementata. Il modello a ciclo singolo, in cui ogni istruzione completa le fasi di fetch, decode ed execute in un unico passo, semplifica la simulazione ma non riflette il comportamento di una microarchitettura reale con pipeline multi-stadio. In un sistema reale, l'introduzione di stadi sovrapposti comporterebbe la gestione di hazard sui dati e di controllo, fenomeni assenti nel modello attuale. Inoltre, la rappresentazione dell'intero stato del core come struttura allocata staticamente limita la scalabilità a valori di NN elevati: aumentando il numero di core, la memoria occupata dal cluster cresce linearmente e l'esecuzione sequenziale dei cicli introduce una latenza simulata proporzionale a NN , rendendo il sistema poco adatto a esperimenti su larga scala senza un refactoring verso allocazione dinamica e, eventualmente, parallelismo a livello di thread sull'host.

6.3 Sviluppi futuri

A partire dall'architettura attualmente implementata, sono possibili diverse direzioni di estensione del framework, sia dal punto di vista strutturale che funzionale.

6.3.1 Estensione della topologia verso reti bidimensionali (Mesh 2D)

L'attuale configurazione lineare può essere generalizzata verso reti bidimensionali di tipo mesh $N \times M$. In questa architettura ogni core disporrebbe fino a quattro vicini attivi lungo le direzioni cardinali, rispetto alla configurazione corrente in cui risultano utilizzati esclusivamente i collegamenti Est e Ovest. Tale estensione richiederebbe la rappresentazione del cluster come matrice bidimensionale `cluster[i][j]` e l'aggiornamento della logica di propagazione dei dati su entrambi gli assi.

Una struttura di questo tipo consentirebbe l'esecuzione efficiente di algoritmi paralleli più complessi, tra cui moltiplicazione di matrici in forma sistolica, convoluzioni su immagini e algoritmi di routing distribuito, sfruttando in modo naturale la località spaziale bidimensionale dei dati.

6.3.2 Integrazione con toolchain di compilazione standard

Attualmente il codice eseguibile sui core deve essere scritto in assembly RISC-V e compilato manualmente tramite una cross-toolchain (ad esempio `riscv64-unknown-elf-gcc`). Un'estensione del framework potrebbe prevedere l'integrazione di uno script di build automatizzato per la generazione di file ELF32 caricabili direttamente dal simulatore, utilizzando flag appropriati come `-march=rv32im`, `-mabi=ilp32` e `-nostdlib`.

Questa estensione ridurrebbe significativamente la complessità di utilizzo del simulatore, permettendo l'impiego di codice scritto in linguaggio C e rendendo il sistema più adatto anche a finalità didattiche e sperimentali.

6.3.3 Pipeline multi-stadio e modellazione degli hazard

L'implementazione attuale assume un modello di esecuzione a ciclo singolo, nel quale ogni istruzione completa le fasi di fetch, decode ed execute all'interno di un unico ciclo di clock. Un'evoluzione naturale consiste nell'introduzione di una pipeline a cinque stadi (IF, ID, EX, MEM, WB), con la modellazione esplicita delle principali tipologie di hazard: strutturali, sui dati (RAW, WAR, WAW) e di controllo.

Tale estensione richiederebbe l'introduzione di registri inter-stadio, logiche di forwarding e meccanismi di stallo. Sebbene aumenti la complessità implementativa, questa modifica permetterebbe di avvicinare il comportamento del simulatore a quello di una microarchitettura reale, consentendo lo studio di fenomeni quali branch penalty e degradazione prestazionale dovuta agli hazard.

6.3.4 Gestione di valori sentinella e tipi di dato estesi

L'attuale implementazione utilizza il valore `0xFFFFFFFF` (interpretato come -1) come sentinella per indicare canali vuoti. Sebbene efficace nel contesto dell'algoritmo di ordinamento implementato, questa scelta limita la rappresentazione di valori validi che coincidono con tale pattern.

Una possibile estensione consiste nell'introduzione di un bit di validità associato a ciascun dato, definendo una struttura del tipo `{uint32_t data; bool valid}` per ogni canale cardinale. In alternativa, è possibile estendere la larghezza dei registri a 64 bit, riservando i bit più significativi alla codifica dei metadati.

Entrambe le soluzioni eliminerebbero la dipendenza dal valore sentinella e garantirebbero la correttezza del sistema per qualunque distribuzione di input.

6.3.5 Strumenti di profilazione e visualizzazione

L'attuale infrastruttura di osservabilità si basa su output testuali a console, sufficienti per la validazione funzionale ma poco scalabili per analisi su larga scala. Un'estensione del sistema potrebbe prevedere l'esportazione delle tracce di esecuzione in formati strutturati come CSV o JSON, contenenti lo stato dei registri, dei canali cardinali e delle istruzioni eseguite a ogni ciclo di clock.

Tali dati potrebbero essere analizzati tramite strumenti esterni, ad esempio script Python con `matplotlib`, per la generazione di visualizzazioni spazio-temporali della propagazione dei dati nella rete sistolica. Questo consentirebbe di osservare in modo diretto il fronte di avanzamento dei valori e il punto di convergenza dell'algoritmo, facilitando sia il debugging sia l'analisi prestazionale.

Appendice A

Codice in C del simulatore RISC-V

A.1 Main

```
1  #include "elf.h"
2  #include "core.h"
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <stdint.h>
6  #include <string.h>
7
8  #define N          4
9  #define MAX_STEPS 200
10
11 uint32_t inOvest_vettore[MAX_STEPS];
12 uint32_t inEst_vettore[MAX_STEPS];
13
14 void run_single_core(RISCV_Core *core) {
15     printf("\n[SINGLE CORE] Premi INVIO per ogni passo. Ctrl+C per uscire.\n");
16
17     while (core->running) {
18         printf("\nPremi INVIO per eseguire il prossimo step...");
19         fflush(stdout);
20         int c;
21         while ((c = getchar()) != '\n' && c != EOF);
22         execute_step(core);
23         print_state(core);
24     }
25     printf("\n[SINGLE CORE] Esecuzione terminata.\n");
26 }
27
28 void run_multi_core(RISCV_Core cluster[], int n, int max_steps) {
29     int step = 0;
30     int core_attivi = n;
31
32     printf("\n[MULTI CORE] %d core in catena. Premi INVIO per ogni ciclo.\n", n);
33
34     while (core_attivi > 0 && step < max_steps) {
35         printf("\n \n");
```

```

36     printf("    CICLO DI CLOCK  %d\n", step);
37     printf("\n");
38
39
40     uint32_t snap_in[N][4];
41     for (int i = 0; i < n; i++){
42         for (int d = 0; d < 4; d++){
43             snap_in[i][d] = cluster[i].in_reg[d];
44
45         }
46     }
47
48
49
50
51
52     for (int i = 0; i < n; i++) {
53         if (!cluster[i].running) continue;
54
55
56         for (int d = 0; d < 4; d++){
57             cluster[i].in_reg[d] = snap_in[i][d];
58         }
59
60         execute_step(&cluster[i]);
61     }
62
63
64     for (int i = 0; i < n; i++) {
65
66         if(i==0){
67             cluster[i].in_reg[OVEST] = inOvest_vettore[step];
68         }else{
69             cluster[i].in_reg[OVEST] = cluster[i-1].out_reg[EST];
70         }
71
72         if(i == n-1){
73             cluster[i].in_reg[EST] = inEst_vettore[step];
74         }else{
75             cluster[i].in_reg[EST] = cluster[i+1].out_reg[OVEST];
76         }
77     }
78
79     // STAMPA interfacce
80     printf("\n Stato interfacce (dopo Fase B):\n");
81     printf("  %-8s | %-8s | %-8s | %-8s | %-8s\n",
82           "Core", "IN_OV", "OUT_OV", "IN_EST", "OUT_EST");
83     printf("  -----|-----|-----|-----|-----\n");
84     for (int i = 0; i < n; i++) {
85         printf("    Core [%d] | %-8d | %-8d | %-8d | %-8d\n",
86               i,
87               cluster[i].in_reg[OVEST],
88               cluster[i].out_reg[OVEST],
89               cluster[i].in_reg[EST],
90               cluster[i].out_reg[EST]);
91     }
92
93
94     printf("\n Valori nel deposito di ogni core:\n ");
95     for (int j = 0; j < n; j++) {
96         printf("Core[%d]: %-4d | ", j, (int32_t)cluster[j].regs[6]);
97     }
98     printf("\n");
99

```

```

100     step++;
101     printf("\nPremi INVIO per il prossimo ciclo...");
102     fflush(stdout);
103     int c;
104     while ((c = getchar()) != '\n' && c != EOF);
105 }
106
107 printf("\n[MULTI CORE] Simulazione terminata in %d cicli.\n", step);
108 }
109
110 int main(int argc, char** argv) {
111     if (argc < 2) {
112         printf("Uso: %s <file.elf> [N_core]\n", argv[0]);
113         return 1;
114     }
115
116     long fileSize;
117     uint8_t* elf_content = load_elf(argv[1], &fileSize);
118     if (!elf_content) return 1;
119     check_elf(elf_content);
120     Elf32_Ehdr* header = (Elf32_Ehdr*)elf_content;
121
122     int n_core = (argc >= 3) ? atoi(argv[2]) : 1;
123     printf("Entry point: 0x%08x | Core: %d\n", header->e_entry, n_core);
124
125     if (n_core == 1) {
126         RISC_V_Core core;
127         init_core(&core, header->e_entry, 0);
128         carica_elf_in_core(&core, elf_content, header);
129         run_single_core(&core);
130
131     } else {
132         if (n_core > N) {
133             printf("Errore: massimo %d core supportati\n", N);
134             free(elf_content);
135             return 1;
136         }
137
138         RISC_V_Core cluster[N];
139         for (int i = 0; i < n_core; i++) {
140             init_core(&cluster[i], header->e_entry, i);
141             carica_elf_in_core(&cluster[i], elf_content, header);
142             printf("Core [%d] inizializzato.\n", i);
143         }
144
145         uint32_t dati[] = {45, 12, 89, 7};
146         int len = 4;
147         int stride = 6;
148
149         for (int s = 0; s < MAX_STEPS; s++) {
150             if (s % stride == 0 && (s / stride) < len)
151                 inOvest_vettore[s] = dati[s / stride];
152             else{
153                 inOvest_vettore[s] = 0xFFFFFFFF;
154                 inEst_vettore[s] = 0xFFFFFFFF;
155             }
156         }
157
158         run_multi_core(cluster, n_core, MAX_STEPS);
159     }
160
161     free(elf_content);
162     return 0;

```

163 }

Listing A.1: main.c

A.2 Core

```

1  #include "core.h"
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdint.h>
6
7
8  static const char* REG_NAMES[32] = {
9      "zero", "ra", "sp", "gp", "tp", "t0", "t1", "t2",
10     "s0", "s1", "a0", "a1", "a2", "a3", "a4", "a5",
11     "a6", "a7", "s2", "s3", "s4", "s5", "s6", "s7",
12     "s8", "s9", "s10", "s11", "t3", "t4", "t5", "t6"
13 };
14
15 static const char* DIR_NAMES[4] = {"NORD", "SUD", "EST", "OVEST"};
16
17 void print_state(RISCV_Core *core) {
18     printf("\n--- STATO CORE [%d] | PC: 0x%08x | INSTR: 0x%08x ---\n",
19         core->core_id, core->pc, core->current_inst);
20
21     for (int i = 0; i < 32; i++) {
22         printf("    x%02d(%-4s)= %-10d", i, REG_NAMES[i], (int32_t)core->regs
23 [i]);
24         if ((i + 1) % 4 == 0) printf("\n");
25     }
26
27     printf("    IN : ");
28     for (int d = 0; d < 4; d++) printf("%s=%-6d ", DIR_NAMES[d], core->
29 in_reg[d]);
30     printf("\n    OUT: ");
31     for (int d = 0; d < 4; d++) printf("%s=%-6d ", DIR_NAMES[d], core->
32 out_reg[d]);
33     printf("\n");
34 }
35
36
37
38
39 void init_core(RISCV_Core *core, uint32_t start_pc, int id) {
40     for (int i = 0; i < 32; i++) core->regs[i] = 0;
41
42     for (int i = 0; i < 4096; i++) core->memory[i] = 0;
43
44     for (int i = 0; i < 4; i++) {
45         core->in_reg[i] = 0xFFFFFFFF;
46         core->out_reg[i] = 0xFFFFFFFF;
47     }
48
49     core->pc = start_pc;
50     core->running = true;
51     core->current_inst = 0;

```

```

52     core->core_id = id;
53 }
54
55
56
57
58
59
60 uint32_t fetch(RISCV_Core *core) {
61     if (core->pc / 4 >= 4096) {
62         printf("Errore: PC fuori dai limiti della memoria!\n");
63         core->running = 0;
64         return 0;
65     }
66
67     uint32_t instr = core->memory[core->pc / 4];
68
69     core->current_inst = instr;
70
71     core->pc += 4;
72
73     return instr;
74 }
75
76
77 DecodedInstr decode(uint32_t instr) {
78     DecodedInstr d;
79
80     d.opcode = instr & 0x7F;
81     d.rd      = (instr >> 7) & 0x1F;
82     d.funct3  = (instr >> 12) & 0x7;
83     d.rs1     = (instr >> 15) & 0x1F;
84     d.rs2     = (instr >> 20) & 0x1F;
85     d.funct7  = (instr >> 25) & 0x7F;
86     d.imm     = 0;
87
88     if (d.opcode == OP_IMM || d.opcode == LOAD || d.opcode == JALR){
89         int32_t imm_i = (instr >> 20) & 0xFFF;
90         if (imm_i & 0x800)
91             imm_i = (int32_t)(imm_i << 20) >> 20;
92         d.imm = imm_i;
93     }
94
95     if (d.opcode == STORE){
96         int32_t imm_s = ((instr >> 7) & 0x1F) | (((instr >> 25) & 0x7F) << 5);
97
98         if (imm_s & 0x800)
99             imm_s |= 0xFFFFF000;
100         d.imm = imm_s;
101     }
102
103     if (d.opcode == BRANCH){
104         int32_t imm_b = ((instr >> 7) & 0x1E)
105             | ((instr >> 20) & 0x7E0)
106             | ((instr << 4) & 0x800)
107             | ((instr >> 19) & 0x1000);
108         if (imm_b & 0x1000)
109             imm_b |= 0xFFFFE000;
110         d.imm = imm_b;
111     }
112
113     if (d.opcode == LUI || d.opcode == AUIPC){
114         d.imm = instr & 0xFFFFF000;
115     }

```

```

116
117     if (d.opcode == JAL){
118         int32_t imm_j = ((instr >> 12) & 0xFF) << 12
119                         | ((instr >> 20) & 0x1) << 11
120                         | ((instr >> 21) & 0x3FF) << 1
121                         | ((instr >> 31) & 0x1) << 20;
122         if (imm_j & 0x100000)
123             imm_j |= 0xFFE00000;
124         d.imm = imm_j;
125     }
126     return d;
127 }
128
129 void execute(RISCV_Core *core, DecodedInstr d) {
130     uint32_t address;
131     uint32_t val1, val2;
132     int branch_taken;
133
134     switch(d.opcode) {
135
136         case LOAD:
137             address = core->regs[d.rs1] + d.imm;
138             switch(d.funct3) {
139                 case 0x0:
140                     core->regs[d.rd] = (int8_t)core->memory[address];
141                     printf("LB x%d, %d(x%d)\n", d.rd, d.imm, d.rs1); break;
142
143                 case 0x1:
144                     core->regs[d.rd] = *(int16_t*)&core->memory[address];
145                     printf("LH x%d, %d(x%d)\n", d.rd, d.imm, d.rs1); break;
146
147                 case 0x2:
148                     core->regs[d.rd] = *(int32_t*)&core->memory[address];
149                     printf("LW x%d, %d(x%d)\n", d.rd, d.imm, d.rs1); break;
150
151                 case 0x4:
152                     core->regs[d.rd] = (uint8_t)core->memory[address];
153                     printf("LBU x%d, %d(x%d)\n", d.rd, d.imm, d.rs1); break;
154
155                 case 0x5:
156                     core->regs[d.rd] = *(uint16_t*)&core->memory[address];
157                     printf("LHU x%d, %d(x%d)\n", d.rd, d.imm, d.rs1); break;
158             }
159             break;
160
161         case STORE:
162             address = core->regs[d.rs1] + d.imm;
163             switch(d.funct3) {
164                 case 0x0:
165                     core->memory[address] = (uint8_t)(core->regs[d.rs2] & 0xFF
166 );
167                     printf("SB x%d, %d(x%d)\n", d.rs2, d.imm, d.rs1); break;
168
169                 case 0x1:
170                     *(uint16_t*)&core->memory[address] = (uint16_t)(core->regs
171 [d.rs2] & 0xFFFF);
172                     printf("SH x%d, %d(x%d)\n", d.rs2, d.imm, d.rs1); break;
173
174                 case 0x2:
175                     *(uint32_t*)&core->memory[address] = (uint32_t)core->regs[
176 d.rs2];
177                     printf("SW x%d, %d(x%d)\n", d.rs2, d.imm, d.rs1); break;

```

```

177         }
178         break;
179
180     case OP_IMM:
181         switch(d.funct3) {
182             case 0x0:
183                 core->regs[d.rd] = core->regs[d.rs1] + d.imm;
184                 printf("ADDI x%d, x%d, %d\n", d.rd, d.rs1, d.imm); break;
185
186             case 0x2:
187                 core->regs[d.rd] = ((int32_t)core->regs[d.rs1] < (int32_t)
d.imm) ? 1 : 0;
188                 printf("SLTI x%d, x%d, %d\n", d.rd, d.rs1, d.imm); break;
189
190             case 0x3:
191                 core->regs[d.rd] = (core->regs[d.rs1] < (uint32_t)d.imm) ?
1 : 0;
192                 printf("SLTIU x%d, x%d, %d\n", d.rd, d.rs1, d.imm); break;
193
194             case 0x4:
195                 core->regs[d.rd] = core->regs[d.rs1] ^ d.imm;
196                 printf("XORI x%d, x%d, %d\n", d.rd, d.rs1, d.imm); break;
197
198             case 0x6:
199                 core->regs[d.rd] = core->regs[d.rs1] | d.imm;
200                 printf("ORI x%d, x%d, %d\n", d.rd, d.rs1, d.imm); break;
201
202             case 0x7:
203                 core->regs[d.rd] = core->regs[d.rs1] & d.imm;
204                 printf("ANDI x%d, x%d, %d\n", d.rd, d.rs1, d.imm); break;
205
206             case 0x1:
207                 core->regs[d.rd] = core->regs[d.rs1] << (d.imm & 0x1F);
208                 printf("SLLI x%d, x%d, %d\n", d.rd, d.rs1, d.imm & 0x1F);
break;
209
210             case 0x5:
211                 if(d.funct7 == 0x20){
212                     core->regs[d.rd] = (int32_t)core->regs[d.rs1] >> (d
.imm & 0x1F);
213                     printf("SRAI x%d, x%d, %d\n", d.rd, d.rs1, d.imm &
0x1F);
214
215                     }else {
216                         core->regs[d.rd] = (uint32_t)core->regs[d.rs1] >> (
d.imm & 0x1F);
217                         printf("SRLI x%d, x%d, %d\n", d.rd, d.rs1, d.imm &
0x1F);
218                     }
219                     break;
220         }
221         break;
222
223     case OP:
224         switch(d.funct3) {
225             case 0x0:
226                 if (d.funct7 == 0x20) {
227                     core->regs[d.rd] = core->regs[d.rs1] - core->regs[d
.rs2];
228                     printf("SUB x%d, x%d, x%d\n", d.rd, d.rs1, d.rs2);
229                 }
230                 else if (d.funct7 == 0x01) {
231                     core->regs[d.rd] = (int32_t)core->regs[d.rs1] * (
int32_t)core->regs[d.rs2];

```

```

232         printf("MUL %d, %d, %d\n", d.rd, d.rs1, d.rs2);
233     }
234     else {
235         core->regs[d.rd] = core->regs[d.rs1] + core->regs[d
236     .rs2];
237         printf("ADD %d, %d, %d\n", d.rd, d.rs1, d.rs2);
238     }
239     break;
240
241     case 0x1:
242         if (d.funct7 == 0x01) {
243             int64_t full_res = (int64_t)(int32_t)core->regs[d.
244             rs1] * (int64_t)(int32_t)core->regs[d.rs2];
245             core->regs[d.rd] = (uint32_t)(full_res >> 32);
246             printf("MULH %d, %d, %d\n", d.rd, d.rs1, d.rs2);
247         } else {
248             core->regs[d.rd] = core->regs[d.rs1] << (core->regs
249             [d.rs2] & 0x1F);
250             printf("SLL %d, %d, %d\n", d.rd, d.rs1, d.rs2);
251         }
252         break;
253
254     case 0x4:
255         if (d.funct7 == 0x01) {
256             if (core->regs[d.rs2] == 0) core->regs[d.rd] = 0
257             xFFFFFFFF;
258             else core->regs[d.rd] = (int32_t)core->regs[d.rs1]
259             / (int32_t)core->regs[d.rs2];
260             printf("DIV %d, %d, %d\n", d.rd, d.rs1, d.rs2);
261         } else {
262             core->regs[d.rd] = core->regs[d.rs1] ^ core->regs[d
263             .rs2];
264             printf("XOR %d, %d, %d\n", d.rd, d.rs1, d.rs2);
265         }
266         break;
267
268     case 0x6:
269         if (d.funct7 == 0x01) {
270             if (core->regs[d.rs2] == 0) core->regs[d.rd] = core
271             ->regs[d.rs1];
272             else core->regs[d.rd] = (int32_t)core->regs[d.rs1]
273             % (int32_t)core->regs[d.rs2];
274             printf("REM %d, %d, %d\n", d.rd, d.rs1, d.rs2);
275         } else {
276             core->regs[d.rd] = core->regs[d.rs1] | core->regs[d
277             .rs2];
278             printf("OR %d, %d, %d\n", d.rd, d.rs1, d.rs2);
279         }
280         break;
281
282     case 0x2:
283         if (d.funct7 == 0x01) {
284             int64_t full_res = (int64_t)(int32_t)core->regs[d.
285             rs1] * (int64_t)(uint64_t)core->regs[d.rs2];
286             core->regs[d.rd] = (uint32_t)(full_res >> 32);
287             printf("MULHSU %d, %d, %d\n", d.rd, d.rs1, d.rs2
288             );
289         } else {
290             core->regs[d.rd] = ((int32_t)core->regs[d.rs1] < (
291             int32_t)core->regs[d.rs2]) ? 1 : 0;
292             printf("SLT %d, %d, %d\n", d.rd, d.rs1, d.rs2);
293         }
294         break;

```

```

284         case 0x3:
285             if (d.funct7 == 0x01) {
286                 uint64_t full_res = (uint64_t)core->regs[d.rs1] * (
uint64_t)core->regs[d.rs2];
287                 core->regs[d.rd] = (uint32_t)(full_res >> 32);
288                 printf("MULHU %d, %d, %d\n", d.rd, d.rs1, d.rs2)
;
289             } else {
290                 core->regs[d.rd] = (core->regs[d.rs1] < core->regs[
d.rs2]) ? 1 : 0;
291                 printf("SLTU %d, %d, %d\n", d.rd, d.rs1, d.rs2);
292             }
293             break;
294         case 0x5:
295             if (d.funct7 == 0x20){
296                 core->regs[d.rd] = (int32_t)core->regs[d.rs1] >> (
core->regs[d.rs2] & 0x1F);
297                 printf("SRA %d, %d, %d\n", d.rd, d.rs1, d.rs2);
298             }
299             else if (d.funct7 == 0x01) {
300                 if (core->regs[d.rs2] == 0) core->regs[d.rd] = 0
xFFFFFFFF;
301                 else core->regs[d.rd] = core->regs[d.rs1] / core->
regs[d.rs2];
302                 printf("DIVU %d, %d, %d\n", d.rd, d.rs1, d.rs2);
303             }
304             else {
305                 core->regs[d.rd] = (uint32_t)core->regs[d.rs1] >> (
core->regs[d.rs2] & 0x1F);
306                 printf("SRL %d, %d, %d\n", d.rd, d.rs1, d.rs2);
307             }
308             break;
309         case 0x7:
310             if (d.funct7 == 0x01) {
311                 if (core->regs[d.rs2] == 0) core->regs[d.rd] = core
->regs[d.rs1];
312                 else core->regs[d.rd] = core->regs[d.rs1] % core->
regs[d.rs2];
313                 printf("REMU %d, %d, %d\n", d.rd, d.rs1, d.rs2);
314             } else {
315                 core->regs[d.rd] = core->regs[d.rs1] & core->regs[d
.rs2];
316                 printf("AND %d, %d, %d\n", d.rd, d.rs1, d.rs2);
317             }
318             break;
319     }
320     break;
321
322     case JALR:
323     {
324         uint32_t return_addr = core->pc;
325         core->pc = (core->regs[d.rs1] + d.imm) & ~1;
326         core->regs[d.rd] = return_addr;
327         printf("JALR %d, %d, %d\n", d.rd, d.rs1, d.imm);
328     }
329     break;
330
331     case BRANCH:
332         branch_taken = 0;
333         val1 = core->regs[d.rs1];
334         val2 = core->regs[d.rs2];
335         switch(d.funct3) {
336             case 0x0: if (val1 == val2) branch_taken = 1;
337                 printf("BEQ"); break;

```

```

338
339         case 0x1: if (val1 != val2) branch_taken = 1;
340                 printf("BNE"); break;
341         case 0x4: if ((int32_t)val1 < (int32_t)val2) branch_taken =
1;
342                 printf("BLT"); break;
343         case 0x5: if ((int32_t)val1 >= (int32_t)val2) branch_taken
= 1;
344                 printf("BGE"); break;
345         case 0x6: if (val1 < val2) branch_taken = 1;
346                 printf("BLTU"); break;
347         case 0x7: if (val1 >= val2) branch_taken = 1;
348                 printf("BGEU"); break;
349     }
350     if (branch_taken) core->pc = (core->pc - 4) + d.imm;
351     printf(" x%d, x%d, %d\n", d.rs1, d.rs2, d.imm);
352     break;
353
354     case LUI: core->regs[d.rd] = d.imm;
355     printf("LUI x%d, 0x%x\n", d.rd, d.imm); break;
356     case AUIPC: core->regs[d.rd] = (core->pc - 4) + d.imm;
357     printf("AUIPC x%d, 0x%x\n", d.rd, d.imm); break;
358
359     case JAL:
360         core->regs[d.rd] = core->pc;
361         core->pc = (core->pc - 4) + d.imm;
362         printf("JAL x%d, %d\n", d.rd, d.imm);
363         break;
364
365     case 0x0B:
366
367         {int dir = d.rs2;
368
369             if (d.funct3 == 0x0) {
370                 core->regs[d.rd] = core->in_reg[dir];
371                 printf("IN x%d, DIR:%d (valore: %d)\n", d.rd, dir, core
->regs[d.rd]);
372             }
373             else if (d.funct3 == 0x1) {
374                 core->out_reg[dir] = core->regs[d.rs1];
375                 printf("OUT x%d, DIR:%d (valore: %d)\n", d.rs1, dir,
core->regs[d.rs1]);
376             }
377
378             break;
379         }
380     }
381
382     core->regs[0] = 0;
383 }
384
385
386 void execute_step(RISCV_Core *core) {
387     if (!core->running) return;
388
389     core->current_inst = fetch(core);
390
391     DecodedInstr d = decode(core->current_inst);
392
393     execute(core, d);
394 }

```

Listing A.2: core.c

```

1  #ifndef CPU_H
2  #define CPU_H
3
4  #include <stdint.h>
5  #include <stdbool.h>
6
7  #define MEM_SIZE 4096
8
9  typedef struct {
10     int      core_id;
11     uint32_t regs[32];
12     uint32_t pc;
13     uint32_t current_inst;
14     uint32_t memory[MEM_SIZE];
15     bool     running;
16
17     uint32_t out_reg[4];
18     uint32_t in_reg[4];
19 } RISC_V_Core;
20
21 void print_state(RISC_V_Core *core);
22 void init_core(RISC_V_Core *core, uint32_t start_pc, int id);
23
24 typedef struct {
25     uint32_t opcode;
26     uint32_t rd;
27     uint32_t rs1;
28     uint32_t rs2;
29     uint32_t funct3;
30     uint32_t funct7;
31     int32_t imm;
32 } DecodedInstr;
33
34 #define NORD  0
35 #define SUD   1
36 #define EST   2
37 #define OVEST 3
38
39 #define OP_IMM 0x13
40 #define LOAD   0x03
41 #define JALR   0x67
42 #define STORE  0x23
43 #define BRANCH 0x63
44 #define LUI    0x37
45 #define AUIPC  0x17
46 #define JAL    0x6F
47 #define OP     0x33
48
49 void init_cpu(RISC_V_Core *cpu);
50 uint32_t fetch(RISC_V_Core *cpu);
51 DecodedInstr decode(uint32_t instr);
52 void execute(RISC_V_Core *core, DecodedInstr d);
53 void execute_step(RISC_V_Core *core);
54
55 #endif

```

Listing A.3: core.h

A.3 Elf

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <stdint.h>
5  #include "elf.h"
6  #include "core.h"
7
8  uint8_t* load_elf(const char* path, long* size) {
9      FILE* f = fopen(path, "rb");
10     if (!f) {
11         perror("Errore apertura file");
12         exit(-1);
13     }
14
15     fseek(f, 0, SEEK_END);
16     *size = ftell(f);
17     fseek(f, 0, SEEK_SET);
18
19     uint8_t* buffer = malloc(*size);
20     fread(buffer, 1, *size, f);
21     fclose(f);
22     return buffer;
23 }
24
25 void carica_elf_in_core(RISCV_Core *core, uint8_t* elf_content, Elf32_Ehdr*
    header) {
26     Elf32_Shdr* sections = (Elf32_Shdr*)(elf_content + header->e_shoff);
27
28     for (int i = 0; i < header->e_shnum; i++) {
29         if (sections[i].sh_type == 1 && (sections[i].sh_flags & 2)) {
30             if (sections[i].sh_addr + sections[i].sh_size > RAM_SIZE) {
31                 printf("Avviso: Sezione %d ignorata (fuori RAM)\n", i);
32                 continue;
33             }
34             uint8_t* ram_ptr = (uint8_t*)core->memory;
35             memcpy(ram_ptr + sections[i].sh_addr, elf_content + sections[i]
    ].sh_offset, sections[i].sh_size);
36         }
37     }
38 }
39
40 void check_elf(uint8_t* content) {
41     if (content[0] != 0x7F || content[1] != 'E' || content[2] != 'L' ||
    content[3] != 'F') {
42         fprintf(stderr, "Errore: Non un file ELF valido\n");
43         exit(-1);
44     }
45     if (content[4] != 1) {
46         fprintf(stderr, "Errore: Atteso ELF a 32 bit\n");
47         exit(-1);
48     }
49     if (content[5] != 1) {
50         fprintf(stderr, "Errore: Atteso Little-Endian\n");
51         exit(-1);
52     }
53 }

```

Listing A.4: elf.c

```

1  #ifndef ELFFILE_H
2  #define ELFFILE_H
3

```

```

4  #include <stdint.h>
5  #include <stdio.h>
6  #include "core.h"
7
8  #define RAM_SIZE 16384
9  #define EI_NIDENT 16
10 #define ELFCLASS32 1
11 #define ELFDATA2LSB 1
12
13 typedef struct {
14     unsigned char e_ident[EI_NIDENT];
15     uint16_t      e_type;
16     uint16_t      e_machine;
17     uint32_t      e_version;
18     uint32_t      e_entry;
19     uint32_t      e_phoff;
20     uint32_t      e_shoff;
21     uint32_t      e_flags;
22     uint16_t      e_ehsize;
23     uint16_t      e_phentsize;
24     uint16_t      e_phnum;
25     uint16_t      e_shentsize;
26     uint16_t      e_shnum;
27     uint16_t      e_shstrndx;
28 } Elf32_Ehdr;
29
30 typedef struct {
31     uint32_t sh_name;
32     uint32_t sh_type;
33     uint32_t sh_flags;
34     uint32_t sh_addr;
35     uint32_t sh_offset;
36     uint32_t sh_size;
37     uint32_t sh_link;
38     uint32_t sh_info;
39     uint32_t sh_addralign;
40     uint32_t sh_entsize;
41 } Elf32_Shdr;
42
43 uint8_t* load_elf(const char* path, long* size) ;
44 void carica_elf_in_core(RISCV_Core *core, uint8_t* elf_content, Elf32_Ehdr*
    header);
45 void check_elf(uint8_t* content);
46
47 #endif

```

Listing A.5: elf.h

Bibliografia

- [1] Krste Asanović et al. The landscape of parallel computing research: A view from berkeley. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-183.pdf>, 2006. Technical Report No. UCB/EECS-2006-183.
- [2] Peter Cheung. Instruction set architecture (RISC-V ISA) – lecture notes. EIE2 Instruction Architectures & Compilers, October 2025. Based on Digital Design and Computer Architecture by Harris & Harris.
- [3] H. T. Kung. Why systolic architectures? *IEEE Computer*, 15(1):37–46, 1982.
- [4] RISC-V International. The risc-v instruction set manual, volume i: Unprivileged isa. <https://docs.riscv.org/reference/isa/>, 2019. Accessed: 2024-05-22.